Anchors and Contracts for Long-Horizon Practice: AC Paradigm V6 Technical Report (TRAE-Tuned)

Sammy Zeng

Dao Wei

2026

[1]Contact email: jop.sammy@163.com ORCID: 0009-0007-4382-0040

[2]This paper is a research outcome of the "Long-Horizon Practice Theory" project, a special support program by the Qianjiang New Town Central Business Industry Community of Shangcheng District, Hangzhou. The community provided critical support in survey coordination, baseline stress-test samples, and real-world business use cases (non-financial support).

Abstract

Failures of large language models in long-horizon tasks are often attributed to insufficient single-shot reasoning capability, but the essence lies in structural fractures caused by non-precomputable sequential dependency chains and the medium constraint. The AC Paradigm (the "Anchor-Contract Human-Machine Collaborative Engineering Paradigm") is proposed precisely as a collaborative engineering middle layer: it translates the continuity of contracts, continuity of state, and independence of evaluation required by long-horizon practice theory into actionable organizational structures, providing the structural lower bound necessary for bounded agents (humans and LLMs) to sustain long-horizon practice. The term "lower bound" here does not refer to a capability lower bound that replaces base model capabilities, but rather to the structural lower bound that long-horizon continuity additionally requires even when the underlying base model already possesses minimal instruction-following, local reasoning, and basic tool-use capabilities. This paper discusses not the ultimate form of the AC Paradigm, but its first complete engineering instantiation on the TRAE IDE—AC Paradigm V6 (TRAE-Tuned). We elaborate on the design principles of core mechanisms including GN-004 Independent Review based on the "veil of ignorance" and subagent dispatch for hedging performance risk, explain why the same set of organizational principles still requires independent tuning around platform interfaces when instantiated in concrete agent tools, and argue for the systematic impact of infrastructure noise on long-horizon practice. Finally, through a combinatorial mathematics case sustained by DeepSeek-v4-pro over 102 hours across 16 rounds of cross-checkpoint iteration, this paper demonstrates how the three-layer structure of anchor–contract–review in the AC Paradigm supports long-horizon closure in the current implementation environment; this case does not constitute sufficient proof of cross-platform universality, but it constitutes the first existence proof of engineering feasibility for this paradigm on a real agent platform.

Keywords: Anchor-Contract Paradigm; Long-Horizon Practice; Human-AI Collaboration; Subagent Dispatch; GN-004 Independent Review; Structural Baseline

Contents

1 Introduction

What the long-horizon practice domain currently lacks is not models that perform better at single-shot responses, but an independently operable layer of collaborative engineering organization. A few relatively engineering-mature solutions are deeply coupled with specific platforms and cannot be detached for reuse; the remaining performance largely comes from the brute-force reasoning capabilities progressively enhanced in base models, which does not correspond to the structural problems of long-horizon practice. The motivation behind the AC Paradigm lies precisely here: abandoning the wait for an abstract "universal agent" and instead transforming the structural requirements of long-horizon practice theory[1] into actionable organizational principles one by one. The AC Paradigm V6 discussed in this paper is the first fully engineered instance of this approach on the TRAE IDE—not the completed final form of the AC Paradigm in its entirety. Genealogically, this approach inherits the tradition of early interactive computing[2] and intelligence-augmenting human–machine collaboration[3], but shifts the focus from single-shot human–machine coupling to cross-session organizational continuity.

Existing benchmark systems measure single-step closure: the common premise behind scores, completion rates, and accuracy is that task information is already complete at the starting point. This is precisely the condition that long-horizon tasks do not satisfy.

1.1 Long-Horizon Is Not About Time Duration

The core definition of long-horizon practice is not time duration, but whether there exists an unpredictably computable sequential dependency chain[1]: the critical input K_{i+1} of step $i+1$ can only be obtained after step i is executed, and the value of K_{i+1} cannot be precomputed before step i is executed.

1.2 Empirical Observation: Three Types of Structural Fractures

In the TRAE implementation environment examined in this paper, the single-shot reasoning of DeepSeek-v4-pro is already sufficiently strong; however, when tasks extend to several hours (e.g., a 102-hour combinatorial mathematics problem), failures still primarily stem from three types of structural fractures rather than insufficient single-shot reasoning:

In long-horizon practice, enlarging the context cannot compensate for uninstantiated truth values that have not yet been generated—when K_{i+1} has not been

generated before step i is executed, a larger context window cannot fill this information gap.

Over multi-day continuous execution, conversation history is not reliable state storage. When restoring context the next day, the previous day's inference chain may have already been scrolled out of the window—what is restored are disordered conversation fragments, not the complete reasoning structure.

Under conditions of executor attention decay, at the judgment point of "whether this path should be continued," the executor tends to choose continuation. Not because the analysis supports it—but because "continue" is an immediately executable operation within the context, while the information needed to judge "should not continue" is absent from the window.

The three types of fractures share the same origin: conversation history, context window, and task closure—these tool protocols are designed assuming single-step closure. The information structure of long-horizon tasks does not obey this assumption.

1.3 What This Paper Answers

This paper takes the TRAE implementation of AC Paradigm V6 as its main thread, but the objects of argumentation are divided into two layers: Sections 3–4 answer what the AC Paradigm is as a collaborative engineering organizational principle and why it is designed as such; Sections 5–9 answer how this organizational principle is implemented on TRAE, what platform constraints it faces, and how these constraints will guide the next stage of evolution. In other words, this paper neither equates the AC Paradigm with a set of TRAE plugins, nor bypasses implementation issues to discuss abstract principles in a vacuum; rather, it attempts to simultaneously articulate the relationship between the "paradigm per se" and the "current implementation profile." Section 8 uses the 102-hour combinatorial mathematics case to demonstrate how this structure actually operates and the boundaries of its evidence within the current implementation environment.

2 Theoretical Bridge: From Long-Horizon Practice to the AC Paradigm

AC Paradigm V6 is not designed in a vacuum—it is a systematic engineering response to Long-Horizon Practice theory[1]. Every structural judgment of that theory (the substrate prerequisite, six necessary conditions, seven categories of engineering compensation) has a corresponding design component in V6. This section organizes these theoretical points item by item, serving as the theoretical anchor for the design explanations in subsequent sections.

2.1 Four Sources of Non-Precomputability

Long-Horizon Practice theory begins from the question "why are critical inputs unavailable right now" and exhaustively derives four mutually exclusive and jointly exhaustive sources:

1. **Epistemic gap (passive)**: knowledge is unavailable until that step is executed. In the 102-hour case, the (q, t)-Catalan equivalence discovery occurred at hour 76—because it depended on the wrapping-structure cognition accumulated across the route attempts of the preceding 75 hours. Without executing those prior routes, this equivalence could not, in our experimental observations, be inferred in advance.

2. **Medium constraint (objective constraint)**: physical limits of the execution medium force task segmentation. The context window ceiling is the most direct form of medium constraint—when the total context of a long-horizon task exceeds window capacity, information must naturally be compressed, selected, and handed off; full-state retention is impossible under existing context window constraints.

3. **Decision fork (active)**: intermediate choices reshape the subsequent search space. In the 102-hour case, the decision near hour 40 to adopt an algebraic route versus a combinatorial route as the primary attack direction directly influenced the reasoning path for the following dozens of hours—an irreversible, all-or-nothing decision.

4. **External drift (independent)**: the external world changes independently. In multi-agent collaboration or platform-dependency scenarios, requirement changes from collaborators, API rate-limit policy adjustments, and other external state changes occur independently of the executor and cannot be anticipated at task launch.

In the TRAE context, medium constraint (context window ceiling) is the most pressing source of pressure, but decision forks and external drift equally threaten value closure—increasing context window size does not resolve these two sources.

2.2 Three-Layer Structural Lower Bound

Long-Horizon Practice theory establishes three mutually irreducible layers of structural lower bound:

- **Contract layer**: immutable constraints that persist stably across checkpoints. In TRAE engineering organization, this corresponds to the data

schemas, interface stubs (`.pyi`), and config schemas under the `public/` directory, as well as the ideological layer within the two core files (the criterion of "what AC Paradigm is").

- **State layer**: current state transmitted across execution cross-sections. In TRAE engineering organization, this corresponds to the spec triad, the note, the `.trae/documents/` change records, and the three handoff structures (engineering process / handoff status / final result).
- **Evaluation layer**: an inspection mechanism independent of the executor. In TRAE engineering organization, this corresponds to the GN-004 independent review (veil-of-ignorance principle, three categories of mandatory triggers, three-level conclusions).

These three layers constitute the "non-removable lower bound" derived from Long-Horizon Practice theory. The question answered by Long-Horizon Practice theory is "what structural lower bounds does long-horizon practice require," while the question answered by AC Paradigm V6 is "how, within TRAE, to build an engineering organization for these lower bounds." AC Paradigm V6's engineering organization is one viable realization built around these three layers, and the following sections demonstrate the internal logic of this realization piece by piece. The handoff relationships from theory to engineering to platform to future are shown in Figure 1.

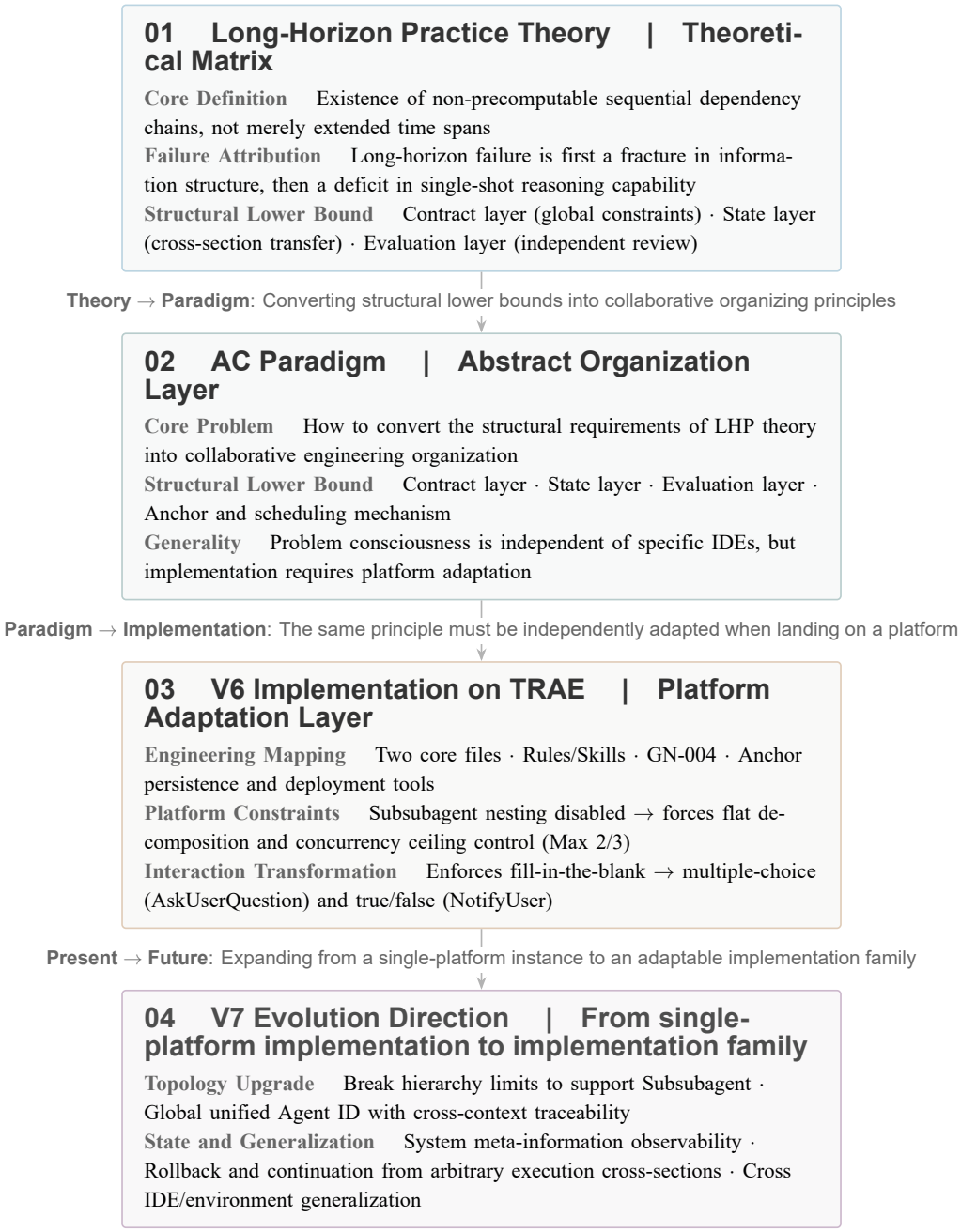

Figure 1: Long-Horizon Practice → AC Paradigm → TRAE Adaptation → V7 Directions Handoff Diagram

3 Definition of the AC Paradigm

The full name of the AC Paradigm is the "Anchor-Contract Human-Machine Collaborative Engineering Paradigm." "AC" is derived from the initials of "Anchor" and "Contract." The anchor system (Section 4 § Anchor System) is responsible for state continuity across execution segments, and the contract system (Section 4 § Three-Layer Contract Definition) is responsible for invariant constraints across roles; together they form the foundational skeleton of the AC Paradigm's engineering organization. Importantly, the AC Paradigm is first and foremost an abstract organizational principle used to translate Long-Horizon Practice theory into operable collaborative structures; the V6 presented in this paper is the first complete engineering instantiation of this principle within the TRAE reference frame, not the full boundary of the AC Paradigm itself.

3.1 Subject: Bounded-Agent Collaboration in Agent-Tool Environments

The direct subject of the AC Paradigm is not "LLMs" or "AI agents," but "bounded agents operating collaboratively within an agent-tool environment (such as the TRAE IDE)." The term "bounded" here applies simultaneously to both humans and LLMs—the two exhibit limitations that are distinct in kind yet share fundamental bounds in terms of information acquisition, depth of reasoning, attentional persistence, and rollback capability.

The engineering practice of V5.3 has validated a key proposition: within an agent-tool environment such as TRAE, humans and LLMs are coupled into a unified whole through the same perception–action loop—humans perceive and constrain system behavior through the IDE interface, AGENTS.md, and spec files, while LLMs perceive and modify the file system through tool calls (Read, Write, RunCommand, etc.). Taken separately, neither constitutes a complete agent: the human lacks tool execution capabilities, while the LLM lacks value arbitration authority.

The AC Paradigm therefore treats human and machine simultaneously as internal constituent elements of a single bounded agent. That the human holds value-judgment authority does not imply "two agents"—just as the separation of kernel-mode and user-mode privileges in an operating system does not imply two operating systems. This is an internal privilege stratification, not a dialogue between two independent agents. This stance situates cognitive processes as activities distributed across the coupled system of humans, tools, and external representations, thereby inheriting the theoretical tradition of Extended Mind[4] and Distributed Cognition[5] in cognitive science—cognition is not confined within the skin but

distributed across all elements constituting the work system.

The engineering practice of V5.3—the operation of the AC Paradigm across 25 Skills, three layers of core files, and 500+ sessions of Long-Horizon Practice—provides direct evidence:

- When the constraints injected by humans through AGENTS.md and the tool-calling behavior of the LLM respond consistently within the same closed loop, the system exhibits stable long-horizon continuity;
- When this closed loop breaks (the human fails to update anchor documents in time, or the LLM fails to read the latest contracts), the system immediately exhibits state drift—and the direction and magnitude of the drift are consistent with the location of the loop break.

3.2 Abstraction-Level Positioning

The abstraction level of the AC Paradigm is independent: it inherits the underlying mechanisms of foundational intelligence theory, reuses the runtime infrastructure of existing agent tools, and exclusively addresses the continuity-constraint problem of Long-Horizon Practice.

- **Foundational Intelligence Theory Layer**: Provides the underlying explanations of "agent interaction" and "bounded rationality" (e.g., Simon's theory of bounded rationality[6], the Observer-to-Agent unified framework[7]), as well as the systematic analysis of "non-precomputable sequential dependency chains" and structural failures from Long-Horizon Practice theory[1]. The AC Paradigm does not investigate the nature of agents; it directly inherits these theoretical foundations.
- **Collaborative Engineering Paradigm Layer (the AC Paradigm)**: Exclusively determines the minimal organizational structure required for long-horizon collaboration. Provides continuity constraints that span multiple execution segments.
- **Runtime/Carrier Layer**: Provides the concrete agent-tool execution environment. The AC Paradigm does not develop independent clients or IDEs; it must rely on existing agent tools as carriers.
- **Domain Practice Layer**: Concrete business scenarios. The AC Paradigm does not contain domain-specific business logic; rather, it provides a general collaborative framework for long-horizon tasks in these domains.

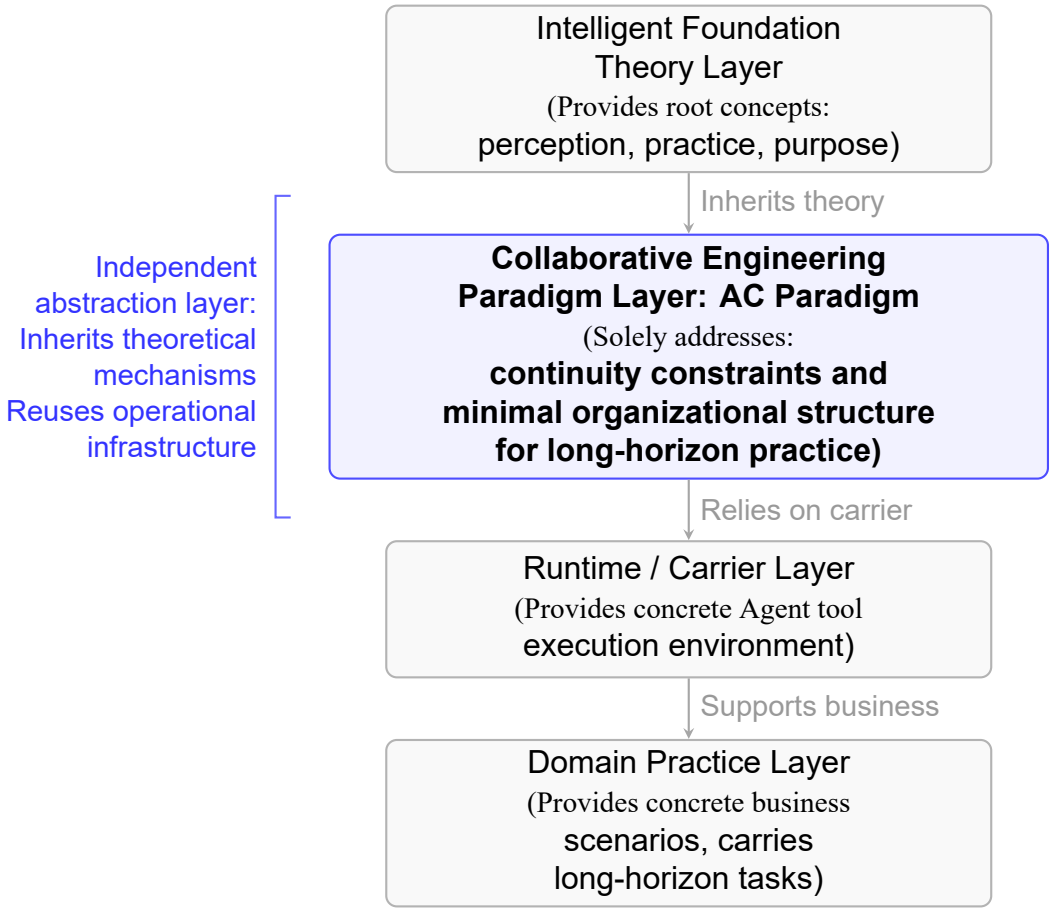

Figure 2: Abstraction-level positioning of the AC Paradigm

This layered taxonomy clarifies the mode of existence of AC Paradigm V6. Although V6 physically manifests as a set of components deeply tuned for TRAE, it is not equivalent at the abstraction level to "a TRAE plugin" or an "operating SOP for a specific tool." It answers a core question that is independent of any particular platform: what continuity structures must exist when the practice of a bounded agent cannot be completed within a single-step closure. A further distinction must be drawn here between two concerns: first, the **abstract independence of the paradigm**—the question answered by AC is not attached to any specific IDE; second, the **portability of the implementation**—the same set of organizational principles, when deployed on different agent tools, still requires independent implementation tailored to each tool's protocol, scheduling interface, and interaction carrier. What V6 has currently accomplished is a complete instantiation of the latter concern within the TRAE reference frame, not a one-shot universal solution across all platforms.

3.3 Root Problem: Structural Mismatch in Long-Horizon Practice

The root problem defined by the AC Paradigm is: what minimal structures are required for Long-Horizon Practice to be sustainably advanced.

This is a lower-bound problem, not an engineering optimization problem. The "lower bound" referred to here is the organizational lower bound for long-horizon continuity, not a capability lower bound that automatically takes effect for arbitrarily weak base models. The AC Paradigm does not substitute for the base model's minimum execution capabilities in single-step instruction following, local reasoning, and basic tool usage; rather, it assumes that these minimum capabilities already exist and further asks: on top of this, if the task spans multiple execution segments, what additional organizational structures are required as a minimum to prevent the system from collapsing. A viable design for sustaining Long-Horizon Practice must at least cover the following three classes of constraints:

1. **Contract Continuity**: Invariant constraints that persist across checkpoints must not be eroded by the convenience of execution processes. Without this layer, after each context truncation, the executor cannot determine "what is correct."

2. **State Continuity**: The current state must be explicitly transmitted across execution segments and must not rely on executor memory or conversation history. Without this layer, upon each context restoration, the executor cannot determine "where we are, why we are here, and what to do next."

3. **Evaluation Independence**: The verification of the preceding two layers must be independent of the executor. Without this layer, executor self-review suffers from a conflict of interest—the structural problem of "being both athlete and referee."

These three constraints jointly constitute the structural boundary of the AC Paradigm: any design that claims to support Long-Horizon Practice must cover all three classes of constraints.

3.4 Core Structure and Boundaries

At the abstraction level, the core structure of the AC Paradigm consists of the following organizational components (detailed relationships among components are described in Section 4):

- **Contract Layer**: Invariant constraints that stably persist across checkpoints, answering "what must not be eroded by execution convenience."
- **State Layer**: Current state transmitted across execution segments, answering "where we are, why, and where to continue from next."
- **Evaluation Layer**: An inspection mechanism independent of the executor, answering "who independently verifies that the preceding two layers have not drifted."
- **Anchor System**: Externalizes contracts and state as stable representations, assuming the functions of reading, handover, and trace retention across segments.
- **Scheduling and Execution Mechanism**: Organizes local task isolation, sequential-relationship control, human–machine signal transformation, and independent review into an operational flow for sustainable practice.

In the V6/TRAE implementation discussed in this paper, the above abstract components are further mapped to a set of concrete engineering artifacts: two core files (the Ideation Layer and the Capability Lock), eight Rules files, eleven Skills, GN-004 independent review, seven categories of anchor systems, and independent supporting components such as `acp-skill-creator-6-1`, the Requirement Structuring Analysis Pipeline, and `deploy-ac-v6-components`. These describe how the AC Paradigm is implemented within the TRAE reference frame, not how the AC Paradigm must exist by definition.

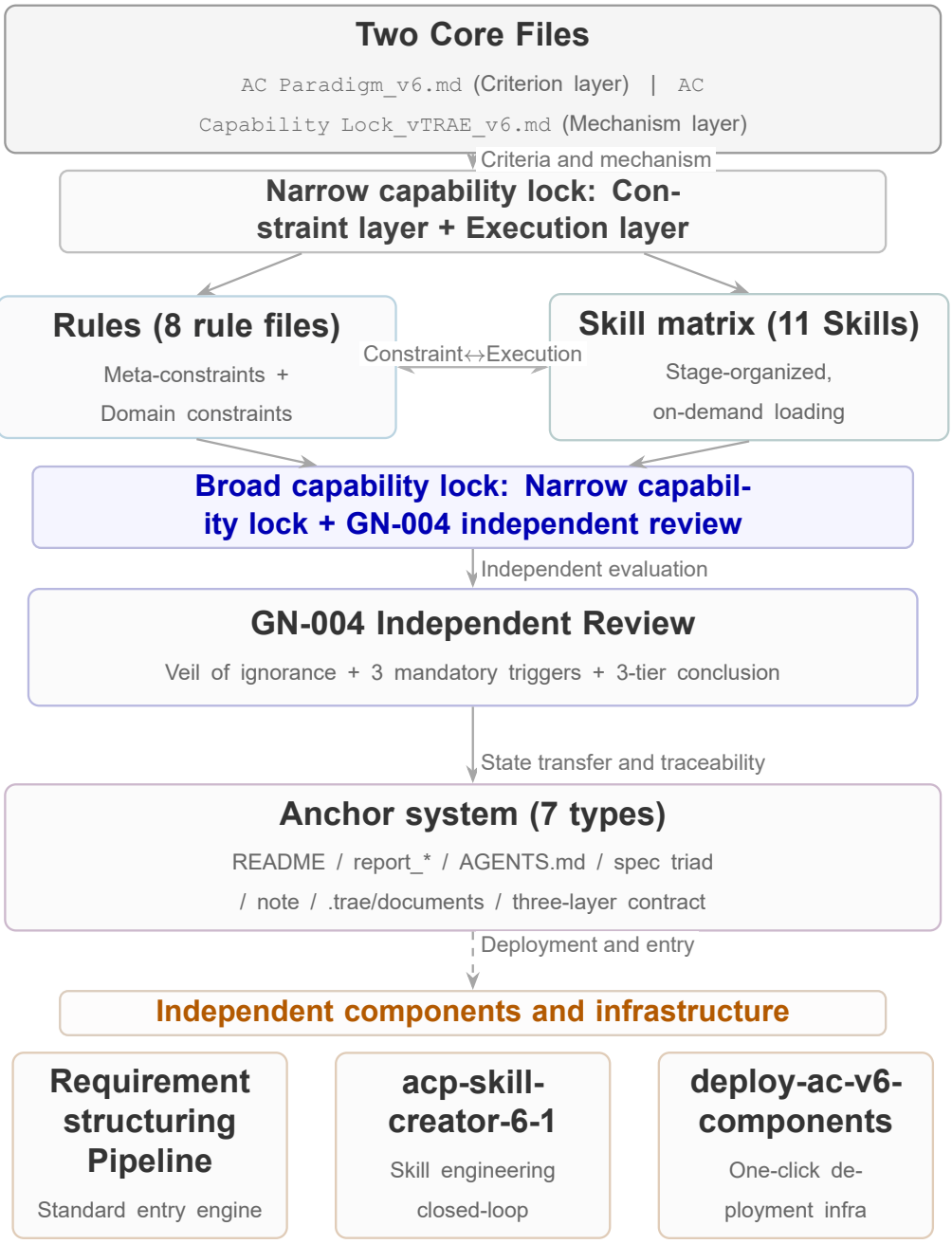

Figure 3: Component panoramic view

The boundary of the AC Paradigm is clearly defined: it does not answer "how LLMs reason" (that is the concern of base models), it does not answer "how specific tasks are executed" (that is the concern of domain practice), and it does not answer "how IDEs should be designed" (that is the concern of platforms). It answers

only one narrow but irreplaceable question: when the practice of bounded agents cannot be completed within a single closure, what organizational constraints are required to make sustained advancement possible.

4 Design Philosophy

Section 3 defined what the AC Paradigm is. This section presents the design decisions of each organizational component: structural rationales, reasons for excluding alternatives, and the derivational relationship with the theoretical foundations of Section 2.

4.1 From Three Layers to Two: Architectural Convergence of Core Files

The core files previously employed a three-layer structure, but continuous engineering exposed the insufficiency of the rules-repository layer as an independent layer: once the capability lock (mechanism layer) was established, the entire content of the rules repository was already absorbed by the capability lock's three-layer structure (meta-constraints / rules-domain constraints / Skill execution). V6 therefore converges it into two layers: `AC_Paradigm_Thought_v6.md` (criterion layer: what AC is) and `AC_Paradigm_CapabilityLock_vTRAE_v6.md` (mechanism layer: how to organize criteria into a sustained practice mechanism). This is not "simplification" but "identifying redundancy and dismantling it."

4.2 The Four Tiers of the Capability Lock

The capability lock is internally organized into four tiers by abstraction level:

1. **Meta-Constraint Tier (rules-0)**: Invariant behavioral rules that remain stably present across projects. This tier carries the direct constraint anchoring points for all 13 conditions (HC-1∼3 / SC-1∼3 / EC-1∼7). Meta-constraints are not "the most detailed rules" but "the rules least dependent on specific scenarios"—they hold regardless of the project's content.

2. **Domain-Constraint Tier (rules-1∼7)**: Instantiates meta-constraints within concrete scenarios. For instance, rules-1 (stage identification) maps SC-2 (ordering constraint) into the 8-stage pipeline, and rules-5 (anchor system) maps HC-3 (state transmission) into specific maintenance rules for the seven anchor types.

3. **Execution Tier (11 Skills)**: Uses Skills as the carrier to define "what tools, in what order, under what trigger conditions" to carry out concrete operations. Skills do not carry rule-based judgments—rules are carried by the Rules layer; Skills only carry "given that the rules are known, how to execute."

4. **Independent Review Tier (GN-004)**: A verification mechanism independent of the preceding three tiers. GN-004 is not an upgraded version of the executor—it does not execute tasks, correct code, or replace the three tiers above. It does exactly one thing: independently verify, against a predefined rubric, whether the outputs of the three tiers are compliant.

The design of these four tiers follows a single principle: each tier performs only verification—not modification—on the outputs of the tier above, and provides only constraints—not substitute execution—on the outputs of the tier below.

4.3 The 13-Condition Encoding Contract: The HC/SC/EC System

Within the above four-tier architecture, the "six logical necessary conditions" and "seven engineering compensation conditions" proposed by Long-Horizon Practice theory are systematically encoded as 13 abbreviated identifiers (HC-1∼3, SC-1∼3, EC-1∼7). This encoding system is a key language that runs through the core design of V6.

4.3.1 Definition of the Encoding and Fracture Types

This encoding system applies a rigorous hard/soft partitioning to the logical necessary conditions based on the "fracture type" induced by their absence, and classifies engineering compensations as a separate category:

- **HC (Hard Conditions, hard-fracture conditions)**: Includes HC-1 (goal closure), HC-2 (checkpoint decomposition), HC-3 (state transmission). These three constitute the physical and logical lower bound of Long-Horizon Practice. When absent, execution undergoes "hard fracture"—the system does not know what to do, does not know how to decompose it, or information completely evaporates at handoff, rendering the practice physically or logically impossible to continue.

- **SC (Soft Conditions, soft-fracture conditions)**: Includes SC-1 (value closure), SC-2 (ordering constraint), SC-3 (unclosed marking). These three constrain the validity of the practice. When absent, execution undergoes

"soft fracture"—the system appears superficially busy and progressing (e.g., marking a wrong branch as completed and continuing), but the entire dependency chain has been built on pseudo-closure, rendering the final result structurally unreliable.

- **EC (Engineering Compensations, engineering compensation conditions)**: Includes EC-1∼7 (e.g., recovery rollback, isolation boundaries, independent evaluation, machine-to-human explanation and consultation). This is the armor layer established to handle high-probability noise such as execution drift, capability deficits, and intention fractures in real-world environments, ensuring that the preceding two categories of logical conditions can actually land in a foundation-model environment replete with such noise.

4.3.2 Why This Encoding Was Established

The design of this encoding contract is by no means merely for abbreviation convenience; it establishes a cross-component "typological language" for the entire AC Paradigm.

The complexity of Long-Horizon Practice demands that mechanisms across multiple dimensions coordinate with one another. Without a unified encoding, the failure of a single stage is difficult to characterize accurately, and the linkages among components would degenerate into vague natural-language descriptions. By introducing the HC/SC/EC prefixes, each condition carries its own declaration of failure-consequence severity and response strategy: once an HC failure is triggered, it indicates a fundamental physical or logical blockage; once an SC failure is triggered, the system must introduce human value judgment or independent evaluation to falsify pseudo-closure; once an EC failure is triggered, compensation mechanisms such as error isolation or consultation with the human must be initiated.

4.3.3 Cross-Component Structural Linkages

This encoding contract becomes the connective tissue running through the entire V6 component matrix; each subsequent core component is structurally linked around it:

- **Delimiting boundaries in the Rules layer**: The meta-constraints (rules-0) are no longer flat, linear provisions but organize constraint entries directly indexed by these 13 encodings. This makes it clear to the executor which are absolutely non-negotiable hard baselines (HC) and which are soft judgments requiring human–machine alignment (SC/EC).

- **Explicit declarations in the Skill matrix**: The first mandatory field of every `SKILL.md` file is `hc_sc_ec_mapping`. At the design stage, a Skill must explicitly declare which specific conditions its operational flow carries, thereby ensuring no structural omission in the mapping from theory to practice.

- **Quantified review in GN-004**: The rubric scoring criteria of the independent review mechanism (GN-004) are directly linked to these encodings. GN-004 does not rely on vague impressions of "looking good or not"; it precisely measures whether the current output has breached HC-3 (has state been reliably transmitted?) or SC-3 (has the unclosed marker been truthfully applied?). Condition mapping transforms review from subjective opinion into reproducible structural measurement.

4.4 The Hierarchical Structure of Rules

4.4.1 Component-Based Decomposition

The eight rules files of V6 (rules-0∼7) are not a simple port of the old rules repository. In past practice, the rules repository existed independently as a long-form instructional document, and there were even attempts to compress large amounts of content into a single rules file, which not only violated the maintainability principle of Long-Horizon Practice but also imposed a heavy contextual burden. V6 implements the AC Paradigm's componentization philosophy by decomposing it into domain-constraint components within the capability lock—each rules file acts as an independent functional constraint that can be independently injected by TRAE's `alwaysApply` mechanism without cross-contamination, thereby making the components maintainable.

4.4.2 Hybrid Strategy for Expressive Carriers

Regarding the expressive carrier of rules, multiple rounds of practice have demonstrated that both pure natural language and pure formal syntax (i.e., machine-executable, structured code) have irreconcilable limitations. V6 ultimately adopts a hybrid strategy of "Chinese as primary, Chinese-English mixed, Python syntax for logical deduction, and other procedural syntax as auxiliary"—a strategy validated across our multiple rounds of engineering practice as the most effective:

- **Trade-offs in the Chinese-English mixed strategy**: Chinese is adopted as the primary language because its information density is high, enabling a 30%∼45% reduction in character count while preserving accuracy of intent,

thereby effectively lowering the contextual burden. However, the root reason that a purely Chinese design cannot be adopted is that coding, action, and function calls inevitably involve English code; under the high-pressure attentional environment of long contexts, forcibly translating or detaching from the English linguistic context would cause a significant degradation in the execution performance of all three.

- **Boundary between formal syntax and natural language**: Using formal syntax such as Python to deduce core logic (e.g., `ec7_action_gate`) effectively eliminates natural-language ambiguity, improves the accuracy of key descriptions, and substantially reduces character count for complex logic. Nevertheless, the core consideration for avoiding purely formal syntax is that certain semantic-level problems (such as value judgments) would become overly rigid if excessively formalized, causing the executor to lose "intelligent flexibility." V6 therefore retains natural language within clearly defined constraint boundaries, granting the Agent the necessary inferential space.

Figure 4 illustrates the responsibility division, cross-reference relationships, and the condition-carrying relationships with Long-Horizon Practice theory across the eight rules files:

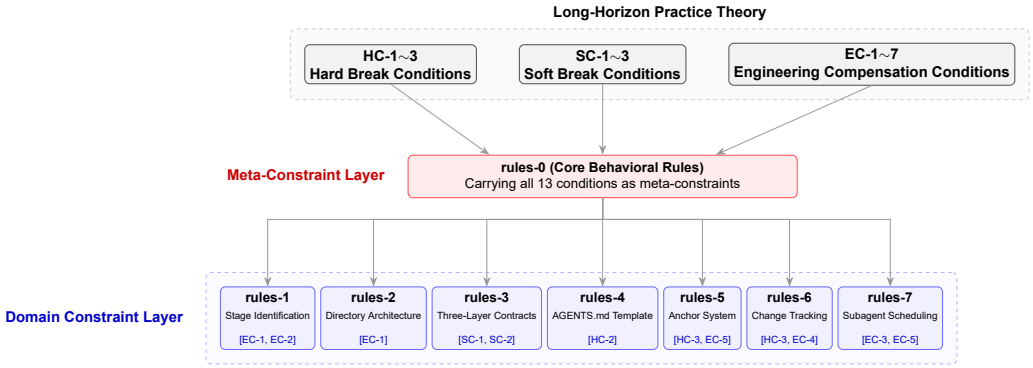

Figure 4: Rules-layer structure: meta-constraints and domain constraints carrying the theoretical conditions of Long-Horizon Practice

4.4.3 Cross-File Mechanism Linkages

EC-3 (independent evaluation) has cross-cutting anchoring points across six rules files—rules-0 (GN-004 trigger mechanism), rules-2 (public/ contract protection), rules-3 (contract test suites), rules-5 (anchor closure verification), rules-6 (document completeness review), and rules-7 (subagent isolation review)—with the

widest coverage, reflecting that independent evaluation is the most central organizational constraint of the AC Paradigm. This is not merely repeating the same sentence across different files, but translating the abstract condition of "independent evaluation" into concrete execution checkpoints in each domain. For example, in rules-2, the evaluation anchoring point is "has the public contract area been tampered with in violation of rules"; in rules-5, the evaluation anchoring point is "do upstream and downstream anchors form a logical closure loop."

The cross-file protection mechanism employs a triple firewall: rules-0 § 4.10 (public/ directory protection) + rules-4 § 4.3 (prohibition on operating in public/) + rules-0 § 4.7.2 (ec7_action_gate tool-call interception). Multi-layer defense prevents any single layer from being bypassed. This design originates from the single-point failure risk discovered during stress testing: relying solely on a single textual rule (rules-4) declaring "modification prohibited" may be ignored when the executor faces explicit instructions (such as "clean up useless files"). V6 therefore introduces dual textual coverage (rules-0 and rules-4 reinforce each other to increase weight) and places a mandatory Python-pseudocode interception layer (ec7_action_gate) on the tool-call path (DeleteFile/Write), upgrading a "rule declaration" into a "pre-operation gate."

4.5 Skill On-Demand Loading Strategy and Progressive Disclosure

4.5.1 Progressive Disclosure and Fractal Isomorphism

In early designs, Skills adopted an always-resident strategy—when the behavioral space is closed and known, exhaustive enumeration is equivalent to completeness. After V6 extended the behavioral space to "Long-Horizon Practice" (an open space), the exhaustive-enumeration strategy failed: the space of possible Long-Horizon Practice forms is infinite, while the context budget is fixed.

V6's Skill system therefore adopts a "progressive disclosure" design, whose core is not "frontmatter always resident $\rightarrow$ body loaded on demand" (name and description are already always visible in TRAE's Skill system), but rather the modular decomposition of Skills into the dual role of "gate + capability."

The underlying logic of this design comes from a core corollary of Long-Horizon Practice theory: construction and execution are fractally isomorphic. The structural design of a practice phase (construction) and the execution operations of that phase (execution) share the same set of abstractions—if independent systems were designed for construction and execution respectively, inevitable semantic drift and contract inconsistency would arise at their boundary. V6 therefore unifies both into a single Skill system: a Skill defines both "what structure this phase is to complete" (construction) and "through what operational sequence to complete

it" (execution).

4.5.2 Hardening Criteria and Boundary Refinement

Regarding hardening criteria, the 11 Skills converge to the minimal set of behavioral abstractions in Long-Horizon Practice that have been validated across multiple rounds and are worth hardening. Excess design would impose contextual burden; insufficient design would fail to cover the structural lower bound of Long-Horizon Practice theory. More importantly, the boundary between each Skill and the rules layer has undergone fine-grained refinement—if a Skill and a rule give structurally contradictory instructions on the same behavior, the Agent's execution performance under high-pressure attention would degrade significantly. Skills therefore do not repeat the declarative constraints already covered by rules, hardening only the "operational procedures" that rules cannot effectively constrain. Furthermore, because construction and execution are fractally isomorphic, the abstractions chosen by a Skill must effectively map both the "construction" and "execution" dimensions at the abstraction level—if an abstraction is adequate on one side but weak on the other, behavior will fracture on the weak side: for instance, a Skill that produces a well-formed structural design but lacks corresponding execution guidance would cause the executor to fill in the operational gaps on its own, thereby deviating from the design intent.

4.5.3 Expressive Carrier: Explicit Code and Explicit Routing

Regarding expressive carriers, practice has shown that Skills should not rely on semantic descriptions for the Agent's base model to "infer" what to do—the action code for tool calls (such as the specific usage of TRAE-specific tools like `AskUserQuestion`, `NotifyUser`, `TodoWrite`) must be explicitly written into the Skill body. Furthermore, when significant sequential relationships exist between flows or behaviors (e.g., s0101→s0102→s0103), Skills should explicitly route to the downstream Skill rather than letting the Agent judge "what to do next" on its own. Practice has validated the risk of missing explicit routing: when a "smart" model, after completing one stage, proactively executes "what it thinks should be done next," procedural justice fails immediately—the Agent skips the routing logic, thereby skipping pre-condition checks and trigger-condition matching, producing anchor displacement and contract drift. From the multi-node perspective of Long-Horizon Practice, drift is not an occasional bug—under conditions of missing explicit routing, it is a systemic pattern that recurs across multi-node Long-Horizon Practice.

4.5.4 Correspondence with Long-Horizon Practice Theory Conditions

This design directly corresponds to the condition system of Long-Horizon Practice theory: HC-3 (state transmission and handoff) lands in the Skill pipeline as each Skill's closure signal and handoff protocol, making every node an explicit state-transition boundary—blocking the propagation of drift at the state layer; SC-2 (ordering constraint) naturally implements phase orchestration through trigger conditions and pre-conditions, with the explicit routing mechanism further ensuring that even if the Agent possesses inferential capability, it must not skip or reverse the execution order; HC-1 (goal closure) and HC-2 (checkpoint decomposition) require that the output of each step conforms to predefined closure criteria—each Skill's closure criteria define the minimum lower bound for "permissible claim of closure," while the fractal isomorphism of construction and execution ensures the self-consistency of closure criteria across both the structural and operational dimensions: what should be completed structurally will not be omitted operationally, and vice versa.

4.5.5 Linkage Design between Skill Fields and the Rules Layer

The linkage between Skills and the rules layer is reflected in the field design of each `SKILL.md`. Each field is not an isolated functional description but the concrete execution-level anchoring point of a constraint within the capability lock:

- `hc_sc_ec_mapping`: Declares which conditions the current Skill specifically covers (e.g., HC-3/SC-2/SC-3), enabling GN-004 to independently verify whether "the conditions a Skill claims to cover are indeed executed in its Action Flow"—this is the serial pathway for EC-3 (independent evaluation) from rules-0 to Skill.

- `State Protocol`: Translates the abstract constraint of "state synchronization and generation specification" in rules-0 into tri-state values (closed / unclosed / currently undecidable), making the executor's progress externally and independently readable at any moment.

- `ec7_trigger_gate` and `ec6_minimum_explanation`: Translates the behavioral transformation mechanisms of EC-7 (open-judgment transformation) and EC-6 (intent communication) from rules-0 into Skill-specific implementations of "under what conditions must human–machine interaction be triggered" and "what minimum information must be provided to the human during interaction."

- `gn004_touchpoints`: Presets review touchpoints at the Skill design stage—what GN-004 should inspect at different phases (three-document set

/ core segments / during execution / pre-delivery). This ensures that independent review is not a retrospective remedy but a structural component embedded at the Skill design stage.

This field design translates the declarative constraints of the rules layer ("must do X") into the operational constraints of the Skill layer ("when doing X, in what format, at what timing, to whom to report"). rules-0 declares legitimacy conditions; Skills define the landing procedures for legitimacy conditions—the two are divided in responsibility but not duplicated.

4.6 The GN-004 Veil-of-Ignorance Principle

GN-004 is the engineering carrier of EC-3 (independent evaluation) within the AC Paradigm. Its design revolves around a core principle—the "veil of ignorance"—validated in engineering practice as a key mechanism for ensuring the baseline fairness of independent review.

4.6.1 Origin and Engineering of the Veil of Ignorance

The "veil of ignorance" originates from the thought experiment proposed by political philosopher John Rawls in *A Theory of Justice*[8]: when people design the basic rules of a society, they must imagine themselves situated behind a "veil"—the veil conceals each individual's concrete social identity, class position, and natural endowments, so that decision-makers have no idea what position they would occupy in the social structure being designed. Precisely because they cannot foresee their own identity, decision-makers are inclined to design a set of rules that is fair to everyone, leaving no one disadvantaged regardless of where they land.

Migrating this principle from social contracts to engineering review yields effectiveness through the exact same mechanism: stripped of the reviewed party's identity, the reviewer is structurally prevented from applying differential standards. Empirical validation confirms this: when GN-004 possesses a priori knowledge of the executor's identity, review quality exhibits bidirectional degradation—if it knows "this is the main thread executing," review standards may be relaxed (an implicit presupposition that "same context, won't be too far off"); if it knows "this is a particular subagent executing," it may presuppose that subagent's output is unreliable and over-review it, causing unnecessary rework. Once identity information is removed, every output is judged solely on its content—the review conclusion is no longer a preset response to "who did it" but a structural measurement of "what was done and how well."

4.6.2 Three Layers of Ignorance: De-Identification from Rules to Skills to GN-004 Itself

The veil of ignorance does not take effect only at the moment GN-004 is launched; it is a systematic "ignorance" design operating across three layers.

Layer One—Internal ignorance of GN-004 by the rules layer and the Skill layer. The rules only describe GN-004's external behavioral interface (launch method, signal types, response rules) and do not expose GN-004's internal review methodology. The same holds for Skills: each Skill's `gn004_touchpoints` field only declares "at what timing GN-004 should be launched to review what content," but must not contain any details about "how GN-004 will conduct the review." This ensures that the executor cannot, through rules or Skills, predict "what standard the output should be reviewed against" and thus cannot preemptively circumvent the review perspective in its output; the reviewer cannot, through the rules layer, predict "how to review" and its review behavior is driven solely by its own rubric. The design outcome of this layer is: even if the executor explicitly knows that GN-004's review basis comes from the 13-condition system of Long-Horizon Practice theory, the executor finds no explicit statement in rules or Skills exposing GN-004's internal review methodology—the executor only knows "what will be reviewed," not "how it will be reviewed."

Layer Two—GN-004's ignorance of its own name. Internally, GN-004 does not know that its external identifier is called "GN-004." This identifier is an external reference label for the rules layer and humans, but GN-004 itself is invoked only by its functional role: it knows its own review methodology (five diagnostic dimensions, rubric scoring criteria, 13-condition fracture verification), and knows that it assumes the role responsibility of "independent verification" within the Long-Horizon Practice workflow, but there exists no self-identifier of "I am GN-004." This design prevents the solidification of reviewer identity: review behavior will not systematically drift because "I am GN-004 so I must be strict" or "I am GN-004 so I must find problems"—it merely faithfully executes the review methodology, without behavioral distortion induced by role labels.

Layer Three—GN-004's ignorance of the reviewed party's identity. When GN-004 is launched, it does not know the executor's identity in advance, understanding responsibilities only through functional descriptions. This prevents the aforementioned bidirectional degradation of review standards.

Together, the three layers of "ignorance" constitute a complete de-identification barrier: the executor does not know how GN-004 reviews (Layer One), GN-004 does not know its own name (Layer Two), and GN-004 does not know who the executor is (Layer Three). Each party is confined within its own cognitive boundary, able only to make or accept judgments based on output content rather than the identity of the output producer.

4.6.3 Three Categories of Mandatory Triggers and Review Density

GN-004's three categories of mandatory triggers (T1/T2/T3) correspond to three structural weak points of Long-Horizon Practice theory—these weak points are precisely the nodes where HC/SC hard and soft fractures among the 13 conditions are most likely to occur:

- T1—Plan Review: Before code output, verify whether the spec/plan/tasks are self-consistent. Blocks directional deviation from entering implementation. Its primary diagnostic targets are HC-1 (is goal closure genuine?) and SC-2 (are there missing or skipped ordering constraints?).
- T2—Key Checkpoint Review: At decision-fork / value-judgment / drift-detection nodes, verify whether the current path deviates from the spec/plan. Its primary diagnostic targets are HC-3 (is state transmission complete?) and SC-3 (are unclosed nodes truthfully marked?).
- T3—Pre-Delivery Closure Review: Before merge, verify whether all outputs satisfy the predefined closure signals. This is a full-scale final inspection of all HC/SC conditions—any fracture not captured in preceding stages will be exposed at T3.

In terms of review density, even a standard, relatively simple Long-Horizon Practice involves at least three GN-004 launches (one each for T1/T2/T3). In genuinely complex tasks—for example, Long-Horizon Practice involving multi-Skill pipelines, multi-subagent parallelism, or cross-decision forks—the actual number of GN-004 launches far exceeds three: in the T2 stage alone, every critical decision fork encountered and every suspicious signal of pseudo-closure may trigger an independent GN-004 review. Although T1 and T3 are nominally one each, if T1 discovers a HARD_BLOCK-level issue, a re-review after correction is mandatory; if T3 discovers residual fractures, it will also retroactively trigger segmented re-reviews. Double-digit or even several dozen GN-004 launches are the norm rather than the exception in complex Long-Horizon Practice.

4.6.4 Default Review Logic: Fracture Inference

GN-004's review logic in its default state is not a generalized "compliance review" but is precisely aligned with the fracture inference of Long-Horizon Practice theory: GN-004's core task is to systematically inspect, one by one, whether the current output triggers the fracture condition of any single HC or SC condition. If HC-1 (goal closure) is not satisfied—i.e., the current output lacks a clear, verifiable closure signal—GN-004 does not issue a "vague" comment and then pass; it directly returns HARD_BLOCK. Similarly, if SC-3 (unclosed marking)

is not truthfully declared—i.e., the executor has an uncompleted subtask that is not marked as "unclosed"—GN-004 also returns HARD_BLOCK rather than an "observation item."

This is the essential difference between GN-004 and general linters or static analysis tools: it does not check compliance in formatting, syntax, or style, but checks "whether this practice will collapse at the next step." Fracture inference means that review conclusions are directly bound to the failure consequences of the 13 conditions—HARD_BLOCK corresponds to hard-fracture conditions (HC failure) or evidence of pseudo-closure (SC failure that cannot be bypassed), SOFT_BLOCK corresponds to soft-fracture boundaries requiring human judgment (significant directional deviation that the human may already be aware of and accept), and Pass means no detectable fracture risk exists on the current cross-section.

4.6.5 HARD_BLOCK and SOFT_BLOCK: The Engineering Semantics of Review Conclusions

GN-004's three-level review conclusions map directly to two explicit engineering blocking signals—HARD_BLOCK and SOFT_BLOCK—together with "Pass" forming a complete tri-state conclusion:

- **HARD_BLOCK (hard block)**: Triggered when HC-level fracture or clear evidence of pseudo-closure is detected. The executor must correct at the blocking location, and after correction the output must undergo a complete independent re-review (not merely "that one point from last time"). HARD_BLOCK is the highest-level block—it means that without correction, subsequent execution will be built upon an already-fractured dependency chain, and the entire body of output will ultimately be untrustworthy.

- **SOFT_BLOCK (soft block)**: Triggered when significant directional deviation, evidence of pseudo-closure, or batch templated confirmation is detected. This category of block must not be bypassed by the executor on its own—it must be delivered to the human for adjudication. Proceed only after the human selects "acknowledged, continue." The design intent of SOFT_BLOCK is to establish a human arbitration channel between "known soft-fracture risk" and "keeping the process moving"—the block does not substitute human judgment but ensures that the human is informed.

- **Warning Pass**: No fracture evidence is currently present, but noteworthy observation items exist. Execution continues after recording the observation items. Observation items are cumulatively tracked at subsequent checkpoints—if these items remain unresolved or worsen at later T2 or T3 stages, they may escalate to SOFT_BLOCK.

- **Pass**: No blocking or warning items whatsoever. All HC/SC conditions on the current cross-section show no fracture signal.

4.6.6 Human-Initiated Review: Extension to the Quality Dimension

The T1/T2/T3 triggers described above are all system-automatic reviews—GN-004 is automatically launched by trigger conditions in the rules layer (such as stage-completion signals, decision-fork events) and executes fracture inference. However, practice has revealed that automatic review alone is insufficient: there exists a class of quality issues in Long-Horizon Practice—not "will it collapse?" but "is it good enough?"—that the structural measurement of automatic review cannot cover. For instance: the output indeed satisfies all HC/SC conditions (it will not fracture), but the abstraction choices, naming strategies, or architectural style may impose additional maintenance costs downstream; the natural-language quality, terminological consistency, or argumentative rigor of Chinese expressions likewise lies beyond the coverage of structural measurement.

The AC Paradigm therefore supports the human explicitly launching GN-004 at any moment and explicitly requesting review of a specific quality dimension. The human may directly instruct in TRAE "have GN-004 specifically review the Chinese expression quality of the current output" or "have GN-004 review whether the current architectural abstraction is over-designed"—at this point, GN-004 superimposes its conventional fracture-inference review with the human-specified quality dimension. The human's explicit quality directive does not replace fracture inference (fracture inference is always GN-004's default baseline) but adds a specific dimension of human concern on top of that baseline.

The five diagnostic dimensions (Contract / State / Execution / Evaluation / Recovery) serve both categories of review simultaneously. In automatic review mode, they are the cognitive framework through which the reviewer inspects fracture inference—the 13 conditions are "what the reviewed subject should satisfy," while the five diagnostic dimensions are "from which angles the reviewer examines the reviewed subject." In human-initiated review mode, the human can anchor quality directives to one or several of these five dimensions, achieving higher-precision targeted review.

4.7 Division of Labor and Elastic Extension of the Anchor System

Anchors are not subsidiary documents; they are externalized constraints that bounded agents rely on to maintain continuity across execution segments. From a cognitive-science perspective, anchor files substantively assume the function of "material anchors"[9]—binding variable reasoning states to stable external representations,

so that cognitive continuity across checkpoints does not depend on the executor's internal memory. V6 defines seven core anchor types, but what they construct is merely the lower bound for sustaining Long-Horizon Practice, not a rigid upper bound.

4.7.1 Seven Foundational Anchors and Engineering Handoff

In the default state, the AC Paradigm relies on seven foundational anchor types for state management and cognitive alignment:

Table 1: Division of labor across the seven anchor types

Anchor	**Positioning**	**Eng. Handoff**	**Update Timing**
README	User-facing entry	No	Project structure/purpose changes
report_*	Judgment sedimentation	No	Phase convergence
AGENTS.md	AI highest-priority rules	No	Capability lock upgrade
Spec three-document set	Task-level constraints	**Yes**	Before task start / before phase switch
note	Cross-segment state transmission	**Yes**	Every Task / phase advancement
.trae/documents/	Change traceability	**Yes**	Every change
Three-layer contract	Public ground-truth source	No	Contract change

The three anchors that assume the engineering handoff function (spec three-document set, note, `.trae/documents/`) are the core carriers for transmitting context between executor and GN-004. They must explicitly contain three segments of information: (1) Engineering process—which tasks/checks have been completed; (2) Handoff status—which task/check is currently in progress (tri-state: closed / unclosed / currently undecidable); (3) Final result—verification conclusions and output inventory for the completed portion.

Among these three handoff anchors, note and `.trae/documents/` have a strict division of labor: the former is responsible for cross-segment state relay, the latter for per-change traceability and correction archiving. The two may cross-reference each other but must absolutely not substitute for each other. The writing meta-principle of note is a functional constraint rather than a format constraint:

every write to note must include the four fields of "where we are / why / unclosed items / continuation entry point." The cognitive-layer partitioning (reading record / diagnostic draft / review record / terminal processing) effectively prevents content from different layers from being intermixed.

4.7.2 Anchor Closure and GN-004 Review Linkage

Anchors are not merely notes that an executor writes for the next executor; they are the legal basis on which GN-004 conducts independent review. At key checkpoints and before delivery, GN-004 is mandatorily required to independently read the currently active spec three-document set, all change records under `.trae/documents/`, and the current note.

GN-004 relies on the three-segment handoff information within these anchor documents to verify the closure status of the Long-Horizon Practice: whether the process is self-consistent, whether the status has been marked with tri-state values, and whether the results have an independently verifiable evidence chain. If the executor attempts to narrow the reading scope (e.g., concealing historical records from GN-004 through conversation summaries), GN-004 will directly flag "attempted restriction of independent reading scope" and trigger a human intervention at the AskUserQuestion level. The pre-delivery closure verification must ensure that all unclosed items have been explicitly marked, rather than silently forgotten in the course of long-horizon execution.

4.7.3 Lower-Bound Design: Anchor Derivation in Real-World Business Contexts

Crucially, the seven anchor types fixed by the AC Paradigm are in fact the **structural lower bound** of Long-Horizon Practice, not the **best-practice upper bound** in real-world business contexts. The AC Paradigm in V6 has not rigidly dictated that "anchor files can only be these"; on the contrary, the paradigm provides ample flexibility, allowing reasonable additional anchor files to be introduced at the practical business level to enhance Agent capability.

In highly complex tasks, depending on the nature of the task, the executor may derive additional specific anchor files during the planning stage (T1). For example:

- **Investigation and decomposition anchors**: When an extremely large SOP system needs to be decomposed into N streamlined audit pathways, a single note file cannot bear the extremely high cognitive load. In such cases, a large number of pre-investigation notes may need to be hierarchically established to fix the conclusions of the first stage, and these additional notes are naturally solidified as new business anchor files during the planning stage.

- **Review-behavior anchors**: In extremely complex system refactoring, GN-004's own high-frequency reviews (dozens of blocks and passes) also generate a large volume of decision context. In such cases, it may be necessary to independently establish a dedicated review anchor file (e.g., `GN004_audit_trail.md`) for GN-004's historical behavioral conclusions, serving as the ground-truth source for tracing "how the system arrived at its current state."

This elastic extension design demonstrates that the anchor system is not a closed dogma. As long as new anchor files can follow the three-segment handoff principle of "process, status, result" and can be effectively read by subsequent executors and GN-004, they should be boldly introduced into Long-Horizon Practice to resist the risk of state collapse brought by complex tasks.

5 TRAE-Specific Tuning

This section does not argue that "AC Paradigm equals TRAE." Rather, it discusses the TRAE-specific implementation of AC Paradigm V6. AC Paradigm, as an abstract organizing principle, answers the question of which continuity structures are required for long-horizon practice. Once these structures land in a concrete agent IDE, however, they must be re-implemented around that platform's tool protocols, interaction carriers, and scheduling boundaries. V6's capability lock name (`vTRAE_v6`), deployment path (`.trae/`), and tool-calling protocol all embed TRAE's platform assumptions for this reason. This is not a retreat from the generality of AC, but a direct exposition of why implementing abstract principles must be specifically tuned.

5.1 Why Targeted Tuning Is Necessary

Different agent IDE and base model combinations exhibit significant differences in fundamental parameters such as minimum instruction following, attention budget decay, and tool-calling behavior. SWE-Agent research has already demonstrated, from the perspective of Agent-Computer Interface (ACI), that tool interface design significantly impacts agent performance[10]—a carefully tuned interface for a specific platform can substantially improve agent reliability, whereas an abstract generic interface behaves unstably in practice. During the V5.3→V6 construction process, the same rules collection showed measurable behavioral consistency differences across different TRAE base models (GPT-5.2, GPT-5.4, etc.)—confirming that "untuned combinations indeed encounter obstacles in practice." This is not a quantitative change of "a slightly higher version works better," but a qualitative change that directly determines whether the combination satisfies the

operational prerequisite. The term **platform-specific adaptation** is used throughout this section to emphasize that what is at stake is not general model fine-tuning, but the adaptation of organizational logic to a concrete agent IDE's tool protocols and scheduling boundaries.

A clarification is warranted, however: non-tuned does not mean completely unusable. Given the current performance of LLMs, even without platform adaptation the abstract principles of AC Paradigm may still allow an agent to complete some tasks at the semantic level—language understanding does not disappear. But "usable in some sense" is not the same as "can be used this way"—the real question is what is lost during cross-platform migration. Platform-specific tools and mechanisms—such as Claude Code's `claude -p` command, TRAE's `AskUserQuestion` structured framework, and the subagent type sets of different IDEs—are entirely inaccessible in a non-tuned environment. The essence of a high-quality Skill is precisely to harden these platform-specific mechanisms into behavior: explicitly writing in tool-calling methods, solidifying behavioral paths, and eliminating the model's room for self-determined interpretation. Once moved across platforms, all such hardening becomes invalid, and only the highly abstract conceptual layer survives migration—the model is thereby forced back into "intuiting" what to do. Practice has repeatedly confirmed that relying solely on model self-inference (rather than explicit programmatic guidance) leads to unpredictable behavior.

This problem is exposed at two levels.

First, at the trigger level. Highly abstract Skill descriptions force the base model to independently figure out "what this Skill is for." A typical failure mode in practice is: the Skill uses designer vocabulary (e.g., "topology-based splitting," "stable surrogate") to describe behavior, but users almost never use these terms in natural language, and LLMs also fail to match them. Empirical evidence comes from the four-factor controlled analysis during acp-skill-creator-6-1 stress testing: s0402 passed all semantic matching tests, whereas s0202/s0203/s0301/s0602 all failed, with the root cause being a disconnect between description vocabulary and users' natural language (users say "Mock/fake data" rather than "stable surrogate," and "module partitioning" rather than "topology-based splitting"). When an LLM cannot recognize "what this Skill is for," it will not trigger that Skill.

Second, at the execution level. Even when a Skill is successfully triggered, if its behavioral description consists of highly abstract principles rather than executable steps, the base model will fill in the behavioral gaps on its own—skipping explicit routing, skipping precondition checks—thereby producing anchor drift and contract drift. Practice has repeatedly confirmed one conclusion: the more abstract a Skill, the more unpredictable the agent's behavior.

Therefore, such platform-specific adaptation is not a decorative optimization, but a prerequisite for ensuring that agent behavior is predictable and auditable.

What is being tuned here is **implementation craft**, not the questions that AC Paradigm itself answers; the generality of the question and the transferability of the implementation are not at the same level.

5.2 From skill-creator to acp-skill-creator-6-1

The stock skill-creator originates from the Claude Code ecosystem, and its design core assumes the Claude model family and Claude Code's toolchain—the Claude Code environment can be used without any configuration because backend assumptions, subagent type assumptions, and behavioral pattern assumptions all point toward the Claude model family.

However, when the stock skill-creator is migrated to TRAE, problems are immediately exposed. TRAE's Skill triggering mechanism relies solely on the `description` field for semantic matching; TRAE provides only two subagent types; and TRAE's `AskUserQuestion` tool implementation differs entirely from that of Claude Code—these differences have no counterpart in the Claude Code ecosystem.

The stock skill-creator's approach to these platform differences is straightforward—abstract to a sufficiently generic principle level. The result is that, apart from extremely abstract principles, it is almost unusable; the base model needs substantial "intuition" to execute, and behavior is highly unstable.

acp-skill-creator-6-1's design philosophy is the opposite: it directly hardens TRAE's platform constraints into Skill behavior, without relying on the base model to "intuit." Three key differences suffice to illustrate:

- The stock skill-creator relies on natural-language communication to advance key decisions—acp-6-1 mandates the use of `AskUserQuestion`'s structured option framework, eliminating the information ambiguity introduced by open-ended questions;
- The stock skill-creator's Skill trigger descriptions are written for designers—acp-6-1's four-factor Description model is specifically optimized for TRAE's LLM semantic matching mechanism, particularly the F2 factor (description vocabulary must overlap with users' natural language), as this is the key variable distinguishing semantic matching success from failure;
- The stock skill-creator's subagent type set includes types that do not exist in TRAE—acp-6-1 hardcodes a prohibition on the `search` subagent, allowing only the two types that TRAE actually provides.

This is not adding a configuration file on top of the stock skill-creator, but a redesign based on TRAE's platform reality. This comparison directly demonstrates

that, without targeted tuning, even conceptually identical tools fail during cross-platform migration. It proves that "platform adaptation is a necessary condition," not that "AC can only exist within TRAE."

5.3 Homologous Evidence from the Deployment Tool

deploy-ac-v6-components[1] is another piece of corroborating evidence for specific tuning: it hardcodes TRAE Agent's dedicated loading URL (distinguishing between the CN edition and the international edition), uses TRAE's project-level convention of the `.trae/` path, and embeds a four-level fallback path to handle the possibility that TRAE's RunCommand path may fail under specific base models or network conditions. The deployment tool itself observes the rules it is meant to deploy—leaving a trace at every step in `.trae/documents/deployment-progress-note.m` which is precisely the self-application of AC Paradigm's anchor traceability principle on the TRAE platform.

5.4 Non-Generality Statement

The current version of AC Paradigm V6 is a TRAE-specific tuned edition. Existing components, when directly migrated to other agent tools (such as Claude Code, Cursor, Windsurf), cannot guarantee the same experience or effectiveness. This non-generality points to **the boundary of current implementation craft**, not to the failure of AC Paradigm's problem awareness as an organizing principle. Clearly defining the applicable boundary has more engineering value than vaguely declaring that "it is already general." Cross-platform generalization will be discussed as a V7 direction in Section 9.

6 Subagents and Media Constraints

6.1 Attention Concentration and Local Compression

In the organizing principles of the AC Paradigm, the subagent strategy is not merely an engineering trick, but a structural means of responding to medium constraint; the V6 implementation on TRAE operationalizes this principle to the greatest extent possible. At the level of a single-step closure, the ReAct paradigm[11] has already demonstrated the effectiveness of interleaving reasoning and action; AC's subagent strategy extends this paradigm from alternating coordination within

[1] AC Paradigm one-click deployment Skill open-source repository: https://github.com/jopsammy/AC-skill-deploy-ac-v6-components

a single-step closure to distributed organization across execution cross-sections. Two mechanisms operate simultaneously:

Elevating local attention concentration. In the main thread context, the executor must simultaneously hold spec constraints, rules constraints, current progress, open items, and cross-task intersections—context crowding directly leads to attention dilution. Placing independent checkpoints or critical independent tasks into subagents, utilizing a cleaner context window (carrying only task instructions + isolation constraints), raises local attention concentration.

High-quality compression. Subagent outputs (analysis reports, review conclusions, proposal outputs) perform high-quality compression of local execution tasks—the subagent completes its reasoning in a clean context, and the condensed conclusion is returned to the main thread. The main thread retains more scheduling headroom without having to consume its own attention budget every time deep analysis is needed.

The theoretical foundation for both of these interpretations can be traced back to the "medium constraint" principle in long-horizon practice theory[1]: the physical limits of the execution medium (context window capacity, attention span) force task segmentation. Subagents do not bypass medium constraint; rather, they transfer the pressure of medium constraint from the main thread into smaller, closed contexts—exploiting the informational advantage that "the smaller the problem, the cleaner the context."

6.2 Hierarchical Strategy for the Attention Budget

Facing medium constraint, the core context management problem immediately emerges: over a sufficiently long temporal span, the total context generated by a long-horizon task will exceed the capacity of a single window—the question is not "can everything be retained" (it cannot), but rather "what to retain, how to compress, and how to ensure the compressed state remains usable."

The subagent strategy directly addresses this constraint:

- The main thread's context window carries only global constraints (two-core file criteria, rules meta-constraints, the current spec/plan) + subagent output summaries—it does not carry the intermediate steps of deep reasoning;
- Each subagent's context window separately carries the complete information required for its task + isolation constraints—once reasoning is complete, the summary can be returned and the context released.

This is essentially "hierarchical management of the attention budget": the main thread's attention is used for global scheduling and constraint maintenance, while

subagent attention is used for local deep reasoning. The two do not share an attention budget pool.

But why is not sharing the budget pool necessarily better? The formal analysis below provides a testable explanatory framework.

6.3 Optimal Performance Projection and Risk Hedging

Based on practical observations from multiple long-horizon tasks, the performance of the same base model can be formally described through a specific coordinate system: let the horizontal axis be time sequence t (task steps or dialogue turns), the vertical axis be the per-turn context increment $c(t)$ fed into the context, and the coordinate origin be the stateless base model. The "performance projection" presented here is better understood as a formal explanatory framework for observations of the current implementation, rather than a universal law that has already been extrapolation-verified across all platforms and all base models.

If we integrate this coordinate system over the interval $[0, T]$ (i.e., compute the projected area under the curve), we define this area as the **Performance Projection (denoted $\mathcal{P}$)**, understood as a cumulative performance integral over time:

$$\mathcal{P} = \int_0^T c(t)\, dt \tag{1}$$

Practice reveals a key regularity: for the same base model, there exists an **optimal performance projection threshold $\mathcal{P}_{\text{opt}}$**, and this threshold is approximately constant, independent of the specific task nature. When the cumulative performance projection $\mathcal{P} \leq \mathcal{P}_{\text{opt}}$, the base model's reasoning performance lies within the optimal range, exhibiting high consistency and reliability (no significant performance difference); once $\mathcal{P} > \mathcal{P}_{\text{opt}}$, i.e., the optimal projection range is breached, the model's instruction-following ability, logical coherence, and local attention undergo a sharp decline.

Comparing the operational characteristics of the main thread versus independent subagents:

- **Main Thread**: As a long-time-sequence model (T_{main} extremely large), although the per-unit-time context increment $c_{\text{main}}(t)$ is relatively small, as the time sequence extends indefinitely, its performance projection $\mathcal{P}_{\text{main}} = \int c_{\text{main}}(t)dt$ increases monotonically without bound, making it highly prone to crossing the $\mathcal{P}_{\text{opt}}$ red line.

- **Subagent**: As a short-time-sequence model (T_{sub} extremely small), even if a large per-turn context increment $c_{\text{sub}}(t)$ is fed in to complete an independent

task, the extremely short integration interval means the final cumulative performance projection $\mathcal{P}_{\text{sub}}$ remains far smaller than that of the main thread, making it extremely unlikely to trigger the threshold breach risk.

Therefore, even for the same base model, over a sufficiently long time sequence, the risk of a subagent breaching the optimal performance projection range is far lower than that of the main thread. This means that, given sufficient context assembly, for local execution tasks, subagent performance is **greater than or equal to** that of the main thread in the vast majority of cases. Fully leveraging the subagent strategy is, at its core, a high-quality hedging strategy against the "optimal performance projection breach risk."

It is worth noting that even if techniques such as context compression and history reset are introduced in the main thread such that $c'(t) < c(t)$, the monotonic growth of the integral $\int c'(t)dt$ over time remains unchanged, and the above conclusion still holds.

This insight fully explains why GN-004 (the independent review mechanism) is highly effective: why, for the same task, the main thread may make errors when executing directly, yet invoking a subagent using the **same base model** for review yields extremely high review quality? This has nothing to do with the performance ceiling of the model we tested; rather, through the mechanism of physical isolation, the integration starting point ($t = 0$) is reset within the subagent, thereby pulling the performance projection for the local task back within the safe zone of $\mathcal{P}_{\text{opt}}$. This demonstrates that, to a certain extent, a sufficient subagent strategy is in fact more efficiently squeezing the physical performance of the model, rather than being purely wasteful engineering overhead.

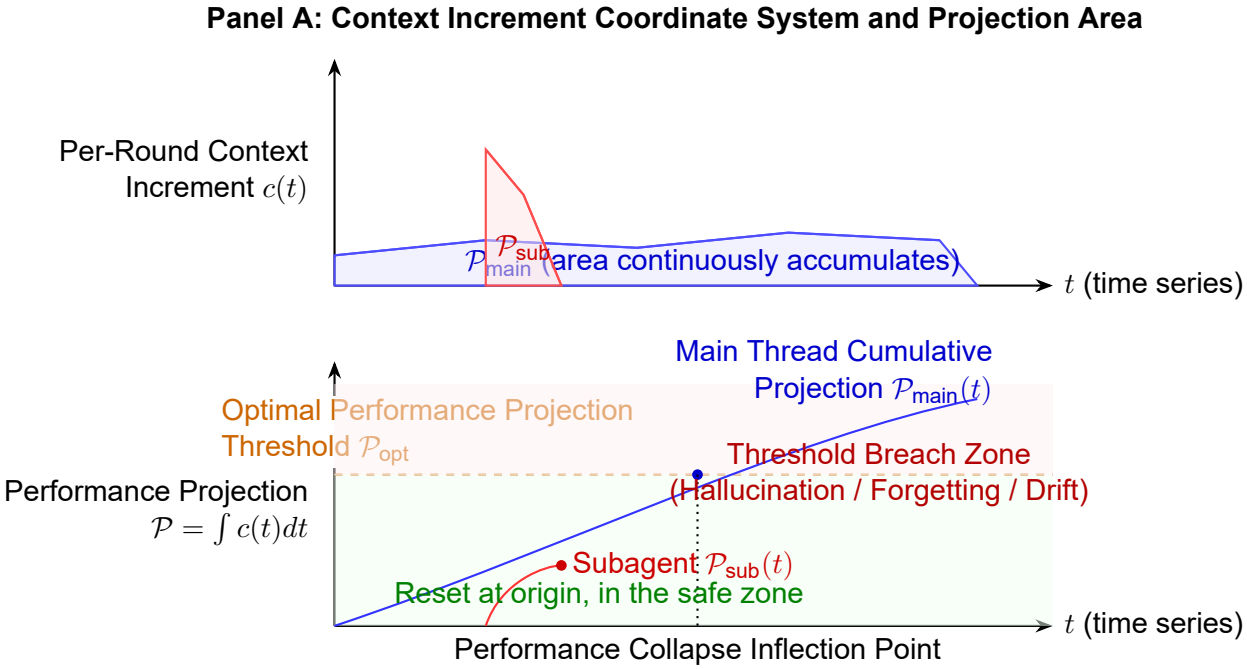

Figure 5: Optimal performance projection ($\mathcal{P}_{\text{opt}}$) and risk-hedging curves for main thread vs. subagent

6.4 Engineering Implications of Parallelism Limits

The parallelism limits of 2/3 ($\leq$ 2 subagents per parallel batch, $\leq$ 3 globally including non-parallel branches) are implementation parameters formed by AC Paradigm V6 within the TRAE reference frame, rather than universal constants in the AC Paradigm definition. These numbers are based on the following three constraints:

- **Platform constraint**: TRAE prohibits nested sub-subagent calls. This means that if a subagent needs to invoke another subagent for independent verification during its execution (e.g., a parallel blind-spot analysis requiring GN-004 review), it cannot do so directly—it can only return to the main thread and have the main thread schedule it. The larger the parallelism count, the more severe the cumulative latency of this "return $\rightarrow$ schedule $\rightarrow$ re-invoke" chain.

- **Attention budget**: Each parallel subagent consumes the main thread's scheduling attention. When the main thread simultaneously manages 4 or more

parallel subagents, its own context begins to crowd—scheduling quality degrades, thereby canceling out the efficiency gains brought by parallelism.

- **Failure isolation**: The larger the parallelism count, the larger the blast radius when a single subagent fails and rollback is required. The 2/3 limit ensures that at most 2∼3 branches are active at any given moment, keeping the operational complexity of failure isolation manageable.

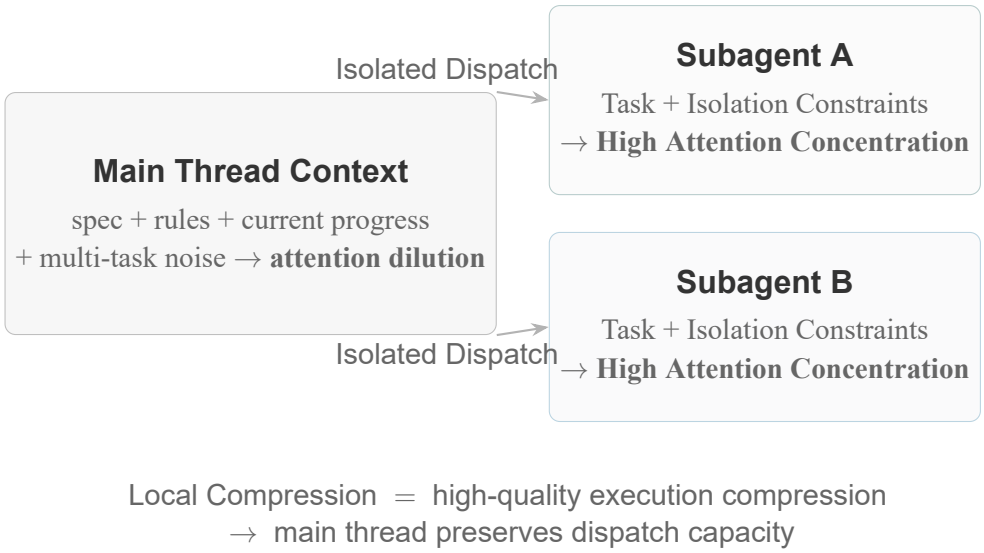

Figure 6: Schematic diagram of subagent attention concentration

7 Infrastructure Noise and Platform Constraints

7.1 Structural Influence of Infrastructure Noise

This section does not discuss the definitional components of the AC Paradigm proper, but rather the environmental noise and implementation constraints encountered when the AC Paradigm is deployed on concrete platforms. In long-horizon practice, infrastructure noise exerts a massive influence on task performance. The infrastructure noise discussed here falls into four categories:

1. **Local machine performance**: CPU and GPU performance directly affect the agent's actual behavior. In long-horizon tasks, a 30% slower single-response time, accumulated over hundreds of tool calls, significantly alters the executor's attention window and the human's tolerance for waiting.

2. **Agent-to-terminal link reliability**: TRAE's agent executes commands and reads output via the terminal. Instability in the terminal link (command execution timeouts, output truncation, encoding issues) is not an occasional event in long-horizon tasks but a recurring source of interference. Every terminal link failure consumes one context recovery—when the failure frequency reaches a certain threshold, the execution rhythm is continuously disrupted.

3. **Base model and toolchain stability**: Different base models (GPT-5.2/GPT-5.4/DeepSeek-V4, etc.) exhibit differing levels of adherence to the same tool-calling protocol. During the V5.3→V6 construction period, it was observed that the same Skill's Action Flow executed completely on one base model but skipped two steps on another—and the skipping behavior was not reproducible. This is not necessarily a "bug" in the base model, but rather a degraded behavior of the tool-calling chain under long-horizon pressure.

4. **Base model API connection stability**: Uncontrollable factors such as API timeouts, rate limiting, and service degradation have limited impact in a single call, but are continuously amplified across hundreds of calls in long-horizon tasks lasting hours to days. Although AC Paradigm V6's anchor-contract mechanism (current-note, spec triptych) guarantees state resumption after an API interruption at the mechanistic level, the frequent interrupt-recovery cycle still continuously erodes the agent's overall performance—each recovery requires extra context window consumption to reload state files, and each interruption breaks the executor's attentional continuity. This "resumable but persistently wearing" effect is a distinctive hidden cost of long-horizon engineering.

The first three types of infrastructure noise admit a unified theoretical interpretation: the carrying premise of long-horizon practice requires the LLM engine to satisfy two hard constraints—minimum instruction following (E3) and non-monotonic decay of the attention budget (H1)[1]. The cumulative effect of infrastructure noise transforms these two constraints in real environments from a binary judgment of "satisfied or not" into a continuous degradation of "still satisfiable at what noise level."

7.2 Empirical Evidence of Infrastructure Noise in the 102-Hour Case

The 102-hour case in Section 8 provides precise empirical evidence of infrastructure noise affecting long-horizon practice. The evaluation environment, as an ex-

ternal execution environment, introduced four types of noise not reproducible locally:

- **Evaluation environment single-core**: v7 OpenMP multi-core parallelism (local 4-core 3.56s) was completely ineffective on the evaluation environment—the evaluation environment executes on a single core, rendering all multi-threaded optimization wasted.
- **Evaluation environment lacks AVX support**: v8 AVX2 compilation failed (no `-mavx2` support); v8b used `__attribute__((target("avx2")))` for isolation, but the evaluation environment's older compiler generated incorrect code—0/19 all wrong.
- **Evaluation environment microarchitectural efficiency approximately 15% lower**: for the same 41.8 billion operations, local Zen4 8.89s vs the evaluation environment >10s. This 15% difference determined whether v11 (already at the pure scalar limit) could pass—ultimately the answer was no.
- **Evaluation environment target attribute incompatibility**: compiler differences in support for specific target attributes led to local pass but environment-wide failure—a classic case of "implicit contract inconsistency between different execution environments."

The common characteristic of these four noise types is: not reproducible locally, only knowable after submitting to the evaluation environment. Each evaluation round consumed $\sim$15 minutes (code modification + compilation + submission + waiting for judging)—5 failed evaluation rounds cumulatively consumed $\sim$75 minutes of engineering time. Within the 102-hour total time span, 75 minutes may seem insignificant, but its impact lies not in the time itself but in the erosion of the executor's judgment caused by "4 consecutive submission failures"—after v7$\rightarrow$v8$\rightarrow$v8b three rounds all wrong, the executor was already inclined to "abandon the hardware optimization route" (in fact, v11 ultimately proved that pure scalar optimization was on the right track—only the judge's environment disallowed it).

7.3 Infrastructure Noise Is Not an Episodic Problem

Infrastructure noise does not naturally disappear with version iteration. Under the framework of long-horizon practice, the parameter space of the execution environment is, in engineering terms, difficult to exhaust before step i is executed—and external drift is defined as "the external world changes independently, and the change is unknowable before step i is executed."

As long as the full parameter space of an external execution environment (judge machine, CI/CD pipeline, third-party API) is not exhaustible before task execution, infrastructure noise remains a structural factor of long-horizon practice—not something that "disappears by doing more," but something that "requires redundancy to be reserved before every task."

AC Paradigm V6's engineering response to infrastructure noise operates at two levels. Crucially, AC's role is not to eliminate the noise itself, but to keep state resumption, degradation paths, and error isolation organizable when noise is unavoidable:

- **Design level**: multi-level fallback paths (the four-level fallback of deploy, the unavailability fallback of GN-004)—not assuming everything is normal, but pre-provisioning "if a certain level fails, how to fall back one level and continue."
- **Mechanism level**: the current-note explicitly records route changes caused by infrastructure noise and "falsified" conclusions—so that the executor, upon cross-day recovery, does not retrace dead ends already falsified by infrastructure noise.

Within this framework, base model API connection stability is a subcategory that requires separate examination: the anchor-contract mechanism guarantees that state can be resumed after an API interruption—the task does not lose all progress due to a single interruption. But it cannot eliminate the continuous drain on the attention budget from the interrupt-recovery cycle: every recovery means additionally loading state files and rebuilding context anchors. In tasks exceeding ten hours, these "recoverable but non-negligible wear costs" substantively reduce the proportion of effective working time. This underscores the necessity of classifying API stability as an independent infrastructure noise source and cannot be classified as "an episodic problem already resolved by the anchor-contract mechanism."

Section 9 will take "local machine performance" and "platform constraints" as the starting point for the V7 direction, discussing the necessity of performance isolation and cross-platform generalization.

8 The 102-Hour Case: Long-Horizon Practice under the AC Paradigm

AC Paradigm V6 has now entered routine operation in real commercial engineering (e.g., cross-cycle system refactoring, multi-module parallel rewriting). However, the actual business codebase cannot be disclosed owing to strict commercial

confidentiality obligations, and the divergent nature of business logic tends to obscure the skeleton of the collaboration mechanism itself. Compared to many long-horizon agent schemes in the industry that remain at the demo stage and rapidly degrade once connected to real high-noise engineering, the AC Paradigm, within the implementation environment discussed in this paper, exists as low-level infrastructure that has withstood the test of daily high-frequency combat.

To provide, within compliance constraints, a logically closed, variable-controlled, and fully reproducible evidentiary specimen, this section deliberately strips away the complexities of the business domain and selects a combinatorial mathematics problem that DeepSeek-v4-pro solved over 102 continuous hours under AC Paradigm V6's TRAE implementation.[2] This is not a coincidental isolated case, but rather an extreme-pressure cross-section of the AC Paradigm's daily operational effectiveness. This section explains not only "why this problem constitutes a rigorous long-horizon practice" but also "why the AC Paradigm's anchor–contract–review structure was able to sustain this cross-6-day continuous problem-solving within the current implementation environment and ultimately succeed." The evidentiary function of this case lies in demonstrating how structural mechanisms work, not in constituting standalone cross-platform statistical proof of universality.

8.1 Case Overview

The problem is an $N = 5000$ combinatorial mathematics / number-theoretic transform (NTT) algorithm problem, with a time limit of 10s and modulus 998244353 (NTT-friendly).

Problem Definition. A bracket string s of length $2n$ composed solely of `(` and `)` is called *legal* if and only if for every prefix the number of left brackets is no fewer than the number of right brackets, and the entire string has exactly n left and n right brackets. For a legal bracket string s, the problem defines two groups of weighting functions:

- alice(s) is the number of subsequences in s equal to `()`—that is, the number of ordered pairs (i, j) such that $1 \le i < j \le 2n$, $s_i =$ `(`, $s_j =$ `)`; $\mathrm{SA}(s) = a^{\mathrm{alice}(s)}$.

- bob(s) is defined through an iterative prefix-batch scanning process; $\mathrm{SB}(s) = b^{\mathrm{bob}(s)}$.

[2] Original research repository (including complete solution records, anchor documents, judge logs, and 35+ artifacts): https://github.com/jopsammy/task1_raw_repo

For each i $(1 \le i \le n)$, let $\mathcal{D}_i$ be the set of all legal bracket strings of length $2i$; the objective is to compute:

$$F_i = \sum_{s \in \mathcal{D}_i} \mathrm{SA}(s) \times \mathrm{SB}(s) = \sum_{s \in \mathcal{D}_i} a^{\mathrm{alice}(s)} \times b^{\mathrm{bob}(s)} \pmod{998244353}.$$

The input is one line of three integers n, a, b $(n \le 5000,\ 1 \le a, b < 998244353)$; the output is one line of n integers $F_1, \ldots, F_n$. The closure criterion is fully mechanically decidable: all 21 test cases passed, runtime $\le$ 10s, memory $\le$ 1GB, single-file implementation, no non-standard degradation strategies—there is no fuzzy intermediate state such as "the direction is correct."

The structural crux of this problem is that Alice's counting (subsequence pairing) and Bob's counting (prefix-batch scanning) are two combinatorially distinct statistics of fundamentally different nature. Unifying them within a single recurrence framework requires a decomposition paradigm capable of simultaneously accommodating tree structure (Alice along Catalan decomposition) and x-sequence structure (Bob along prefix scanning)— and this paradigm could not be anticipated at task launch; it could only be fixed after exhausting multiple candidate routes and verifying them one by one.

DeepSeek-v4-pro solved continuously from May 20 to May 25, 2026, under the AC Paradigm, completing 16+ rounds of judge iterations, proceeding from O(N^3) Python OOM to final full 21/21 pass with the v15-b two-dimensional adaptive NTT implementation.

These 6 days are not "a very hard algorithm problem that happened to take a long time"—its solution path contains a typical non-precomputable sequential dependency chain. The two key breakthrough points occurred at the 76th hour and the 86th hour respectively (not at the 1st hour), because they depended on cognitive accumulation that could not have been obtained in advance without executing to that point. 13 failed mathematical routes, 5 completely different decomposition paradigms—the final breakthrough came not from stronger single-shot reasoning, but from state accumulation and blind-spot discovery across checkpoints. The following subsections address the key questions required for design self-justification in turn.

8.2 Why It Is a Long-Horizon Practice

We first establish the conceptual framework, then unfold the case timeline.

8.2.1 Non-Precomputable Sequential Dependency Chain: The Structural Barrier of Wrapping

Among the 13 failed routes (Routes A∼K + L-DP order-reduction A/B/C), 5 completely different decomposition paradigms—tree decomposition, x-seq decomposition, c-based recurrence, L(i) sequence DP, generating functions—all encountered a common structural obstacle at the wrapping step. The wrapping operation $A \to (A)$ requires a complete height function: computing all height values of the current Dyck word. The key input for step $N+1$ (the complete evaluation of the height function on the given word) can only be obtained after step N has been executed, and the dimension of this input is O(2^k), incompressible to low dimensions. Any attempt to precompute the wrapping result using DP states of O(1) or O(k^c) dimension fails due to information-dimensional mismatch.

This is a precise instance of the "non-precomputable sequential dependency chain" typical of long-horizon practice: the input for step $N+1$ cannot be statically folded before step N is executed. Wrapping is not a computation-volume problem—expanding the context window cannot fold information that has not yet been generated. The common cause of death for 13 routes and 5 paradigms was not insufficient intelligence, but a structural information gap: the global nature of wrapping means that local information is structurally insufficient to close the next step—increasing computation volume cannot fill this information gap.

8.2.2 Both Key Breakthroughs Occurred at Execution-Dependent Epistemic Gap Positions

(q,t)-Catalan equivalence discovery (Hour 76): By item-by-item comparison of the bracket-string weights defined in the problem with standard geometric quantities of Dyck paths, the equivalence of the problem to a specific instance of (q,t)-Catalan numbers was discovered at Hour 76[12-13]. This equivalence relation was not "written into the problem statement from the start"—it depended on the wrapping-structure cognition accumulated during the first 75 hours of route attempts. Without executing those prior routes, this equivalence was structurally unknowable prior to actual execution.

NTT blind-spot discovery (Hour 86): After pushing the pure-scalar optimization version v1→v11 to the limit (O(N^3/6) reduced from 77s to 8.89s), a quantitative calculation established that under the single-core 10s constraint of the evaluation environment, the pure-scalar route could not complete all 20.8 billion dot products. This "physically infeasible" quantitative evidence triggered critical blind-spot awareness—the executor realized that the pure-scalar route must be abandoned in favor of algebraic dimension reduction. Subsequent blind-spot analysis discovered the q-factorial separation + NTT convolution path, converting the

O(N^3) bottleneck to NTT convolution.

The temporal positions of the two breakthroughs: the (q,t)-Catalan equivalence depended on the failure-experience accumulation of the first 75 hours; the NTT blind spot depended on the engineering-limit squeezing of Hours 60–86. Both are epistemic gaps of the execution-dependent type—at task launch, it was impossible to predict at which hour what would be discovered.

8.3 Case Timeline

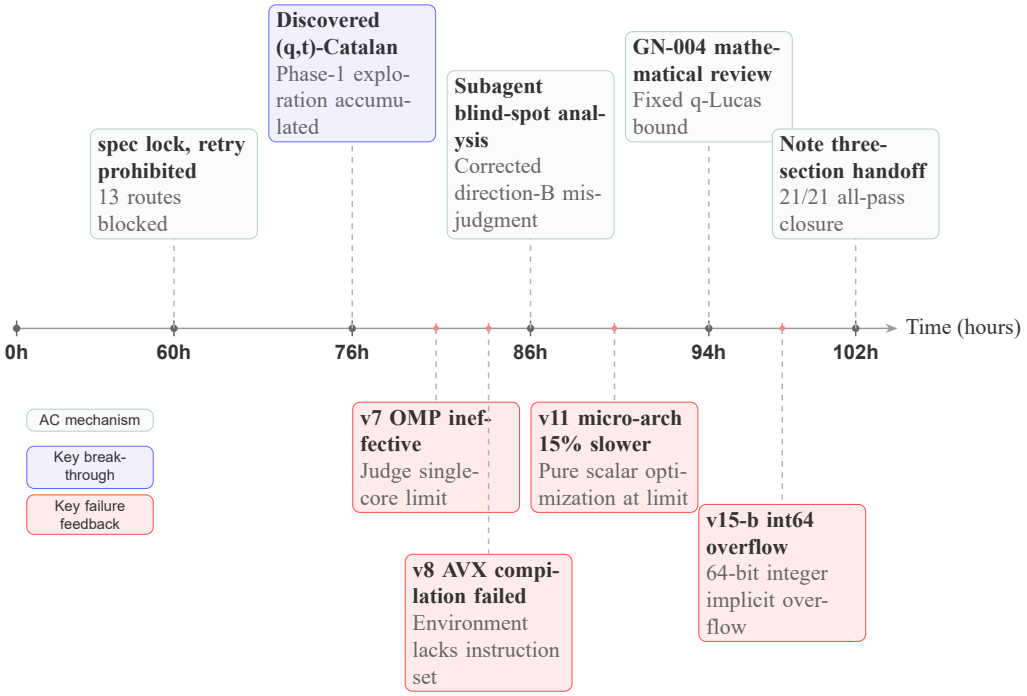

Figure 7: Progression chain of the 102-hour case

Figure 7 displays the key nodes of the 102 hours on a timeline. The following supplements the structural background of each phase in chronological order.

8.3.1 Phase 1 (0–60h): Exhaustive Route Enumeration

Under the anchoring of `spec.md`, the executor attempted 13 mathematical routes. The closure criterion for each route was predefined in `tasks.md`, and the failure root cause of each route was recorded in `current-note.md`. 13 routes collectively pointed to the structural infeasibility of wrapping— this conclusion was

written into `spec.md`'s "falsified facts" at Hour 60 and marked as "forbidden retry routes."

8.3.2 Phase 2 (60–86h): Engineering-Limit Squeezing

After confirming the mathematical routes were blocked, the executor pivoted to engineering optimization routes: v1 (naïve $\sim N^3/6$ Python OOM) $\rightarrow$ v2 (C++ port, 77s) $\rightarrow$ v3/v4 (triangular matrix + diagonal optimization) $\rightarrow$ v5/v6 (4-way/8-way ILP unrolling) $\rightarrow$ v7 (OpenMP multi-core parallelism—but evaluation environment single-core, ineffective) $\rightarrow$ v8/v8b (AVX2—but evaluation environment unsupported, 0/19 all wrong) $\rightarrow$ v9/v10 (loop interchange + register optimization) $\rightarrow$ v11 (32-way ILP, 8.89s, pure-scalar limit reached).

v11's 8.89s passed locally, but exceeded the 10s limit on the evaluation environment (microarchitecture efficiency roughly 15% lower). This cross-platform discrepancy directly triggered the blind-spot breakthrough at Hour 86: the executor quantitatively confirmed that the pure-scalar route was physically infeasible.

8.3.3 Phase 3 (86–102h): Algebraic Dimension Reduction and Final Convergence

At Hour 86, blind-spot analysis discovered the q-factorial separation + NTT convolution path (v14), converting the O(N^3) bottleneck to NTT convolution. v14 achieved 7/21 correct, 14/21 WA on first judge submission—root cause: q-factorial zero-divisor bug (Fact[i]=0 when a has small multiplicative order, leading to 0/0 division). Fixed to a dual path: NTT + v11 fallback.

At Hour 94, GN-004 conducted two rounds of independent mathematical review of v15's q-Lucas decomposition formula: Round 1 discovered an index shift; Round 2 discovered a borrow omission—both errors were corrected before C++ implementation. v15 judged 19/21, 2/21 WA—signed int64 overflow. v15-b fixed to unsigned __int128, 290/290 cross-validation all pass, 21/21 judge all pass.

8.4 Why the AC Structure Can Sustain It in This Implementation

Within our experimental scope, no base model was observed to successfully solve this problem under bare-connection API conditions—the problem's own non-precomputable sequential dependency chain means that any base model, stripped of structural constraints, would irreversibly deviate from the correct direction within 10 rounds due to cross-checkpoint context fracture. Yet once the AC Paradigm's anchor–contract–review three-layer structure is loaded in the TRAE implementation discussed in this paper, DeepSeek-v4-pro completed 16+ rounds of judge iterations

within 102 hours and ultimately achieved closure. This result supports the judgment that "structural constraints are necessary," but does not independently prove that AC is the only feasible path; it more accurately illustrates that the three-layer structure identified by the AC Paradigm can indeed form effective support on a real agent platform.

More specifically, a distinction must be drawn here between two categories of factors: the first is the organizational structure provided by the AC Paradigm, such as anchors, forbidden retry routes, independent review, and isolated dispatch; the second is the execution carrier and noise background provided by the current implementation environment, such as TRAE's toolchain, the DeepSeek-v4-pro base model, judge-machine constraints, and platform interaction boundaries. The case's validity depends on the co-occurrence of both, but what truly constitutes the core argument of this paper is how the former sustains long-horizon continuity amid the real-world noise of the latter.

Anchor documents sustain cross-checkpoint state continuity: Four anchor documents— `spec.md` (proven/falsified facts + forbidden retry routes + GATE table), `tasks.md` (dependency DAG + closure criteria), `checklist.md` (three-layer verification), `current-note.md` (1550+ lines of research log + obsoletion index)— enable the executor (and human observers) to avoid re-reading all code each time context is restored across the 6-day span. The obsoletion index (9 overturned conclusions + overturn reasons + overturn dates) prevents new executors from referencing already-falsified intermediate conclusions.

Forbidden retry routes prevent resource waste: `spec.md` § III explicitly lists 9 forbidden retry routes, each accompanied by its failure root cause. This ensures that the executor, upon resumption across days, does not re-attempt already-falsified paths— "the structured sedimentation of negative knowledge" is a key differentiator of the AC Paradigm from ordinary prompt engineering.

GN-004 independent review prevented mathematical errors from entering implementation: v15's q-Lucas decomposition formula underwent two rounds of independent mathematical review by GN-004— Round 1 discovered an index shift ($\binom{r_0+k_0}{r_0}_a$ should be $\binom{r_0+k_0-1}{r_0}_a$), Round 2 discovered a borrow omission (when $k_0 = 0$, $r_0 = 0$, a borrow $\binom{r_1+k_1-1}{r_1}$ is generated and must not be omitted). Both errors discovered are boundary conditions easily overlooked in derivation; had they directly entered C++ code, they would have wasted at least 2 judge rounds ($\sim$30 minutes of engineering time).

Subagent-isolated blind-spot analysis discovered breakthrough directions missed by the main executor: The 0524 blind-spot analysis corrected the misjudgment that "Direction B (NTT) is a dead end"— the key insight was that the same technique (NTT) has different applicability at different positions (L(i) DP vs. Garsia-Haglund inner layer), but this distinction was suppressed in the main execu-

tor's context by the "Direction B is dead" label. The 0525 blind-spot analysis falsified the conclusion that "q-Lucas inevitably leads to constant explosion"— discovering that residue-layer summation is also cross-correlation and NTT-reducible.

The value of the two blind-spot analyses lies in their independence: unconstrained by the main executor's existing cognitive framework, they re-examined the problem with a clean context.

8.5 Which Mechanisms Are at Work

Table 2: Evidence of AC mechanisms at work in the 102-hour case

AC Mechanism	Specific Use in the Case	Evidence Location
`spec.md`	22 proven facts (all executors must observe), 13 falsified routes (with common cause of death), 9 forbidden retry routes (prevent backtracking), GATE reference table, 17-round judge feedback summary	`spec.md` § I ~ § VI
`tasks.md`	Subtask dependency DAG, per-task closure criteria, version evolution record (v1→v15-b), closed-route table	`tasks.md` S0-1 ~ S0-7
`checklist.md`	Global exit gates (GATE-1 ~ 5), mathematical correctness 67-item checklist, performance benchmarks (v11/v14/v15 comparison), compliance C1–C6	`checklist.md` all sections
`current-note.md`	1550+ lines of research log: obsoletion index (9 items), GATE reference table, judge feedback history, proven/falsified facts, complete records of 13 routes, version evolution, artifact inventory (35+ files)	`current-note.md` § I ~ § XIX
GN-004	Two rounds of mathematical review (q-Lucas index shift + borrow omission), both completed before C++ implementation	`current-note.md` L1476–L1491
Subagent (blind-spot analysis)	0524 analysis corrected "Direction B dead end" misjudgment → v14 breakthrough; 0525 analysis falsified "q-Lucas constant explosion" → v15 breakthrough	`blind-spot-analysis-note0524/`
Three-segment handoff structure	`current-note.md` § XVII explicitly annotates the three segments of engineering process / handoff status / final result; closure notes include operation-count measurements + physical feasibility analysis	`current-note.md` L1294–L1355

Table 2 lists the AC mechanisms that actually functioned in the case and their evidence of use. The blind-spot analyses were completed in the form of independent note files in this practice, without using a formal subagent dispatch framework—but this does not diminish the critical role played by blind-spot analysis as an independent review mechanism: both blind-spot analyses were completed in clean contexts, unconstrained by the main executor's existing cognitive framework, and respectively discovered the NTT breakthrough direction and the q-Lucas dimension-reduction path that the main executor had missed.

9 Limitations and V7 Direction

AC Paradigm V6 is not the endpoint. What this section enumerates are not all "theoretical limitations" of the AC Paradigm as an abstract organizing principle, but rather 7 engineering boundaries exposed by its current implementation within the TRAE reference frame. Among them, Item 1 (TRAE specialization) is an engineering task AC itself has yet to complete, and can be advanced in V7 via a platform adaptation layer; the remaining 6 items—subsubagent nesting, agent id traceability, system meta-information observability, state snapshot rollback, performance isolation, and tool call quotas—expose the capability boundaries of the current TRAE platform on these dimensions. The direction of these limitations is clear, but the path to breaking through them involves capability evolution at the platform level, which exceeds AC's engineering scope as a higher-level organizing paradigm. In other words, V7 is not a self-negation of V6, but the next step from "single-platform implementation" toward "implementation family expansion."

9.1 Limitations → V7 Direction Mapping

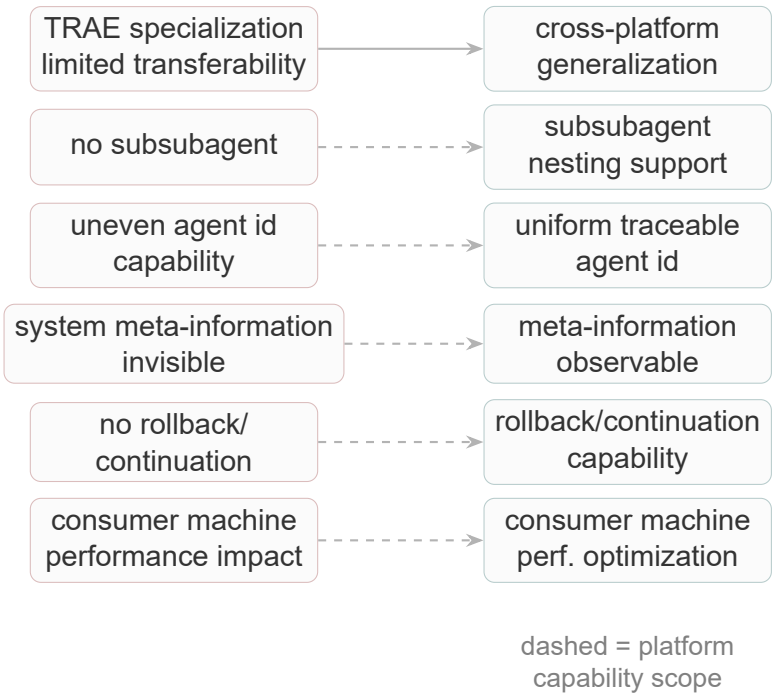

Figure 8: TRAE Constraints → V7 Direction Mapping

Figure 8 lists the 7 limitations and their corresponding directions. Item 1 (TRAE specialization) corresponds to the as-yet-unfinished cross-platform adaptation problem of AC itself; the 6 items marked with dashed lines expose the capability boundaries of the current platform on dimensions such as subagent scheduling, meta-information observability, state management, performance isolation, and resource quotas—the directions are clear, and breakthroughs require capability provisioning from the platform side. What they constrain is the current implementation capability of AC on TRAE, and they do not constitute a refutation of the organizational problem that AC itself proposes. The following expands each limitation item by item: current status, root cause, and corresponding resolution direction.

9.1.1 TRAE Specialization → Cross-Platform Generalization

Current Status: All components of AC Paradigm V6—the two core files, 7 Rules, 11 Skills, GN-004, deploy-ac-v6-components, acp-skill-creator-6-1—are all tailored specifically to TRAE's agent tooling. Hard-coded path conventions (.trae/), TRAE-specific Agent URLs, the four-factor description model, parallelism caps

(2/3), and TRAE CLI calls within the four-level degradation path—these design choices make V6 "install-and-go" on TRAE, but prevent direct migration to other agent platforms (e.g., Cursor, Cline, Copilot Chat, etc.).

TRAE specialization is a design choice of V6: Section 5 has already argued that, based on AC paradigm thinking and long-horizon practice theory, independent testing and tuning must be carried out around each "agent tooling + base model" combination. Untuned combinations will encounter practical obstacles, and there are corresponding theoretical reasons for this.

V7 Direction: Cross-platform generalization. Abstract the TRAE-coupled parts of V6 into a platform adapter, decoupling AC's core organizational logic (capability lock, Rules hierarchy, GN-004 review protocol, anchor system) from the tool names, path conventions, and agent scheduling mechanisms of specific platforms. Generalization is not "removing TRAE-specific tuning"—it is "doing equally deep tuning for each platform, while sharing the same organizational logic."

9.1.2 No Subsubagent → Subsubagent Nesting Support

Current Status: TRAE does not allow launching subsubagents within a subagent. This is a platform-side guard mechanism to prevent runaway nesting from causing token explosion. However, from the AC paradigm's perspective, this constraint directly obstructs the organizational ceiling of "maximally exploiting subagents."

The specific impact is most pronounced in two scenarios: (1) GN-004 review chain—in the current review→fix→re-review loop, the fix step is executed on the main thread rather than in the isolated context of independent review, reducing the sustained intensity of the "veil of ignorance." (2) Blind-spot analysis chain—if a problem discovered by a subagent requires further decomposition into sub-analyses, the current approach can only pass intermediate results back to the main thread, which then assigns new subagents, increasing context contamination and scheduling overhead.

Resolution Direction: The platform side needs to provide limited-depth subsubagent nesting scheduling capability. Nesting isolation falls within the scope of the platform's scheduling mechanism—setting a nesting depth cap (e.g., 2 levels), with nested subagents inheriting the parent's core rules and tasks constraints but not inheriting the current execution state. The degree of openness of the nesting mechanism and the granularity of token consumption control belong to the platform side's design domain.

9.1.3 Uneven Agent ID Support → Unified Agent ID Traceability

Current Status: In the TRAE environment, only the built-in GPT models (5.2, 5.4) consistently expose agent ids. Other base models (e.g., DeepSeek-v4-pro) cannot reliably obtain agent ids. The root cause attribution is currently undecidable—it may be a TRAE platform constraint (only exposing system meta-information for specific base models), or it may be a capability unique to GPT models (the model side can perceive and relay this meta-information).

In the 102-hour case, blind-spot analysis existed as independent note files, not through the formal subagent scheduling framework, making agent ids untraceable—so the judgment that "this analysis was completed in an independent subagent context" cannot be independently verified. The completeness of the subagent scheduling ledger (rules-0 § IV-11) is also constrained by the uneven availability of agent ids.

Resolution Direction: The platform side should provide unified observable output of agent ids. Agent ids are produced by the platform scheduling layer and belong to the platform's information provisioning scope—regardless of which base model is used, they should be standardized and output as a stable observable signal, making them a standard meta-information field for subagent scheduling. Once the platform uniformly outputs this signal, AC can incorporate it as a standard field in the subagent scheduling ledger.

9.1.4 Invisible System Meta-Information → Observable System Meta-Information

Current Status: Apart from GPT models, the main thread cannot reliably observe system information, including: whether Skills have been loaded, the list of currently loaded Rules, the list of available tool types, and the scheduling status of current subagents. This problem is coupled with the uneven agent id support: invisible system meta-information means the main thread has insufficient observability into the system state machine, which affects scheduling precision and anomaly recovery capability.

In the subagent scheduling strategy discussed in Section 6 (attention concentration + local compression), scheduling decisions presuppose the main thread's real-time awareness of "which subagents are currently running, which Skills are loaded." When this premise is unreliable, scheduling decisions can only rely on "estimation" rather than "observation"—this accumulates as systematic drift in long-horizon tasks.

Resolution Direction: The platform side should open system meta-information query interfaces. AC needs a set of platform-agnostic query interfaces to obtain: the list of currently active Rules, the list of loaded Skills, available tool types and current tool call quotas, and the list and status of active subagents. Currently, this

information is managed internally by the platform. When the platform opens corresponding query interfaces, AC can build more precise scheduling logic on top of them.

9.1.5 Missing Rollback/Continuation Capability → State Machine Snapshots + Flexible Rollback

Current Status: TRAE currently does not support sub-task-level state machine snapshots and flexible rollback. Once infrastructure noise occurs during subagent execution (terminal timeout, output truncation, tool call failure), the subagent's context state is unrecoverable and can only be restarted from scratch. It is also impossible to perform a local rollback to a specific completed checkpoint—under the current platform, rollback operations require re-executing the entire task segment.

In the 102-hour case, the 5-round failure of the evaluation environment caused a large amount of engineering time to be consumed in "re-submitting to the evaluation environment" rather than in "algorithm design." If sub-task-level state snapshots existed, the executor could have created a snapshot at v11 (pure scalar limit) and, upon failure of the v14 (NTT) approach, quickly rolled back to v11's engineering baseline, without needing to re-align the entire context from spec.md.

AC's own anchor system (the triplicate handoff structure of spec/note/trae/documents) has already compensated for this problem at the design level: they provide state recovery capability at the "cross-session" level (enabling continuity when reopening a project the next day). But they cannot compensate for real-time rollback "within the same session"— the current recovery granularity of the anchor system is at the "task checkpoint" level, not at the "subagent internal execution state" level.

Resolution Direction: The platform side should provide sub-task-level state snapshot and rollback capability. AC's anchor system has already partially compensated for state recovery needs at the "cross-session" level. Platform-level real-time rollback involves storage, context serialization, and scheduling state recovery, which belongs to the platform architecture domain—in the tradition of distributed systems, state snapshots and globally consistent state recovery already have a mature theoretical foundation[14]. Migrating this framework into agent execution contexts, creating recoverable state snapshots at task checkpoints (including anchor state, Task closure checklist, subagent execution records), would allow the executor to roll back to the nearest snapshot point when subsequent steps fail. Once the platform provides snapshot capability, AC's anchor system can form a complementary relationship with it.

9.1.6 Local Machine Performance → Consumer Hardware Optimization

Current Status: Local machine CPU and GPU performance directly affects the overall actual performance of the agent. This is not a marginal factor where "environmental noise is negligible"—Section 7 has already demonstrated the structural impact of infrastructure noise from both theoretical and empirical perspectives. In the 102-hour case, the ∼15% performance difference between the local Zen4 and the judge machine directly determined whether an optimization path was viable.

The more critical issue is that the current TRAE platform's performance on consumer hardware is far below the hardware ceiling. Empirical measurements have exposed this problem [3]: with 3 parallel SubAgents running (no local inference, API linked to remote models), CPU average is only ∼30% but peaks can reach 99%, GPU occupies 327 MB but utilization is only 4%. Sustained CPU average usage is dominated by `trae-helper` and `ai-agent` daemon processes (15–25%), CPU peaks are dominated by a Recalculate Style bottleneck caused by the lack of DOM virtualization in the Chat Panel (30–60% peak contribution); GPU memory occupation stems from texture accumulation in the Electron rendering pipeline under UMA architecture. Even more counterintuitive is that the "Trae accelerates after exiting WeChat" phenomenon is attributed to two mechanisms—reduced V8 GC frequency and the cessation of Windows Working Set trimming—indicating that the platform's own operational mechanisms (rather than hardware compute power) are the dominant factor in the current consumer hardware performance bottleneck.

In other words, it is not that consumer hardware is insufficiently powerful, but that the platform has not yet fully exploited the CPU, GPU, and memory resources that consumer hardware already provides.

Resolution Direction: The platform side should carry out consumer hardware performance optimization at the operational mechanism level. Specifically: introducing DOM virtualization in the Chat Panel to eliminate the Recalculate Style bottleneck, optimizing the CPU usage of `trae-helper` and `ai-agent` daemon processes, optimizing GPU memory management in the Electron rendering pipeline, and proactive avoidance strategies for cross-process resource contention (e.g., Working Set competition between WeChat and Trae). Once the platform fully harnesses the CPU, GPU, and memory performance of consumer hardware, AC Paradigm's scheduling strategies and degradation paths can operate on a more stable execution baseline, without needing to rely on cloud migration or hardware upgrades to solve performance problems.

[3] See "Analysis of CPU/GPU Resource Consumption in Trae IDE Parallel SubAgent Scenarios and the WeChat Exit Acceleration Effect" (May 2026), Sammy Zeng.

9.1.7 Hard Tool Call Quota Cap → Elastic Tool Call Quota Expansion

Current Status: TRAE imposes a hard cap on the number of tool calls per single conversation—defaulting to 200, with a maximum of only 500 after manual adjustment. In the context of AC Paradigm long-horizon practice, even a single simple long-horizon task routinely exceeds a thousand tool calls; the 102-hour case involved far more than that. After the quota is exhausted, agent performance degrades significantly—attention is forced to shift from the task itself to a survival strategy of "how to complete the remaining steps with fewer tools," and execution quality decays as the quota approaches its limit. More seriously, when the base model's quota is exhausted, it is forced to use CLI commands or scripts to forcibly bypass TRAE's tool call channel—this not only reduces the agent's organizational controllability, but also causes the systematic tool-based scheduling that AC depends on (subagent launching, GN-004 review, Skill invocation, etc.) to degrade at the quota boundary into primitive command-line operations that are difficult to trace.

The quota boundary also prevents the "attention concentration + local compression" scheduling strategy from covering the full task cycle—the scheduler must prematurely close decisions near the quota limit, unable to schedule subsequent subagents at the optimal rhythm. This couples with the scheduling precision problem discussed in Section 6: scheduling precision is already constrained by invisible system meta-information, and the hard quota cap further compresses the scheduler's available decision space.

Resolution Direction: The platform side should provide elastic tool call quota expansion. Raise the tool call cap per single conversation to a level suitable for long-horizon tasks, or provide an automatic quota renewal mechanism. Quota expansion involves the platform's resource allocation policy and cost control, and belongs to the platform side's design domain. AC can clearly specify the demand boundary for tool call volume in long-horizon tasks, and optimize scheduling strategies on top of it once the platform provides elastic quotas.

9.2 Attribution and Boundaries

Table 3 summarizes the attribution and direction status of the 7 limitations. Apart from Item 1, which falls within AC's own engineering scope, the remaining 6 items expose the capability boundaries of the platform on corresponding dimensions.

Table 3: Summary of Limitation Attribution and Direction Status

Direction	Attribution	Status Description
Cross-platform generalization	AC engineering scope	Platform adapter architecture is discussable; specific platforms await one-by-one validation
Subsubagent nesting	Platform constraint	Nesting depth and token management strategy belong to platform scheduling mechanism domain
Unified agent id traceability	Platform constraint	Agent ids are produced by the platform scheduling layer; belongs to platform information provisioning domain
System meta-information observability	Platform constraint	Meta-information is internally managed by the platform; belongs to platform observability domain
State machine snapshots + flexible rollback	Platform constraint	Involves platform storage and scheduling state recovery; belongs to platform architecture domain
Consumer hardware optimization	Platform constraint	Agent operational mechanisms need deep optimization for consumer CPU/GPU/memory
Elastic tool call quota expansion	Platform constraint	Quota policy and resource management belong to platform resource allocation domain

Among these 7 directions, Item 1 is an engineering task that V7 can actually advance; breakthroughs for the remaining 6 require capability provisioning from the platform side. AC's current strategy is: maximize organizational efficacy under existing constraints through the anchor system (§ 4), scheduling strategy (§ 6), and fallback paths (§ 7), while recording clear demand signals for each platform constraint—so that when corresponding capabilities become available on the platform side, AC can rapidly adapt based on the existing demand signals and design accumulation.

10 Acknowledgments

We thank the Qianjiang New Town Central Business Industry Community of Shangcheng District, Hangzhou for providing critical support in survey coordination, baseline stress-test samples, and real-world business use cases. We thank DeepSeek for providing Deepseek-v4-pro base model support; the 102-hour stress run was completed by this base model under the Anchor-Contract Human-Machine Collabo-

rative Engineering Paradigm (AC Paradigm V6). We thank the Hangzhou Synergistic Referral Platform Community (SRO) for providing substantive use case support. We thank members of the Comet Project for providing technical support. We also thank the many members of the AI-Zion community for their inspiring discussions and valuable suggestions, as well as TRAE and Cherry Studio for providing critical IDE environment and technical support.

References

[1] ZENG Q, WEI D. Long-Horizon Practice: The Structural Baseline Beyond Single-Shot Capability[Z/OL]. 2025. https://doi.org/10.5281/zenodo.20470866.

[2] LICKLIDER J C R. Man-Computer Symbiosis[J/OL]. IRE Transactions on Human Factors in Electronics, 1960, HFE-1(1): 4-11. DOI: 10.1109/THFE2.1960.4503259.

[3] ENGELBART D C. Augmenting Human Intellect: A Conceptual Framework[M]. Stanford Research Institute, 1962.

[4] CLARK A, CHALMERS D J. The Extended Mind[J/OL]. Analysis, 1998, 58(1): 7-19. DOI: 10.1093/analys/58.1.7.

[5] HUTCHINS E. Cognition in the Wild[M]. MIT Press, 1995.

[6] SIMON H A. A Behavioral Model of Rational Choice[J/OL]. Quarterly Journal of Economics, 1955, 69(1): 99-118. DOI: 10.2307/1884852.

[7] LIU F, LV B, LIU Y. From Observer to Agent: On the Unification of Physics and Intelligence Science[Z/OL]. 2024. https://www.preprints.org/manuscript/202410.0479/v1.

[8] RAWLS J. A Theory of Justice[M]. Revised. Harvard University Press, 1999.

[9] HUTCHINS E. Material Anchors for Conceptual Blends[J/OL]. Journal of Pragmatics, 2005, 37(10): 1555-1577. DOI: 10.1016/j.pragma.2004.06.008.

[10] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[C]//Advances in Neural Information Processing Systems: vol. 37. 2024.

[11] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations (ICLR). 2023.

[12] GARSIA A M, HAIMAN M. A Remarkable q,t-Catalan Sequence and q-Lagrange Inversion[J/OL]. Journal of Algebraic Combinatorics, 1996, 5(3): 191-244. DOI: 10.1023/A:1022476211638.

[13] HAGLUND J. The q,t-Catalan Numbers and the Space of Diagonal Harmonics: vol. 41[M]. American Mathematical Society, 2008.

[14] CHANDY K M, LAMPORT L. Distributed Snapshots: Determining Global States of Distributed Systems[J/OL]. ACM Transactions on Computer Systems, 1985, 3(1): 63-75. DOI: 10.1145/214451.214456.